

Misi: Simulación de Gato con Generación Procedural de Ciudad en Godot 4

Guía Completa de Conocimientos

Autores:

César Aarón Perales Rosales
Melisa Karen Rivera ALagon
Fabryzzio Jossue Meza Torres
Flavio Jose Tipula Meza

Universidad: UTEC



Índice general

I	Introducción	5
0.1	Descripción del Proyecto	6
0.2	Objetivos del Documento	6
0.3	Conocimientos Previos Asumidos	6
0.4	Conocimientos NO Asumidos	6
0.5	Estructura del Documento	7
0.5.1	Arquitectura de Dependencias	7
0.6	Tiempo Estimado	7
II	Fundamentos	8
1	Fundamentos de Godot 4	9
1.1	Instalación y Configuración	9
1.2	Estructura de Escenas y Nodos	11
1.2.1	Herencia e Instanciación	12
1.2.2	Carga Dinámica de Escenas	13
1.3	Programación en GDScript	13
1.3.1	Variables Exportadas	14
1.3.2	Señales y Métodos	15
1.3.3	Ciclo de Vida (_ready, _process)	16
1.4	Sistema de Materiales	17
1.4.1	Creación de Materiales PBR	18
1.4.2	Mapeo UV y Texturas	20
1.5	Sistema de Física de Godot	21
1.5.1	Cuerpos Rígidos (StaticBody3D)	21
1.5.2	Sistema de Raycast	22
1.5.3	Detección de Colisiones	24
2	Matemáticas 3D Básicas	26
2.1	Operaciones con Vector3	26
2.1.1	Producto Escalar y Vectorial	26
2.1.2	Normalización de Vectores	27
2.1.3	Cálculo de Distancias y Ángulos	28
2.2	Transformaciones 3D	28
2.2.1	Espacios Global y Local	28
2.2.2	Rotaciones y Cuaterniones	29
2.2.3	Interpolación Lineal (lerp/slerp)	29

2.3	Geometría Computacional	30
2.3.1	Triangulación de Delaunay	30
2.3.2	Diagrama de Voronoi	31
2.3.3	Recorte de Polígonos	31
2.3.4	Punto en Polígono	31
III	Sistema de Comportamiento del Gato	33
3	Sistema de Locomoción Procedural	34
3.1	Control de Movimiento mediante Nodo Objetivo	34
3.2	Cinemática Inversa (IK) para las Patas del gato	34
3.3	Orientación Automática del Cuerpo	35
3.4	Simulación de la Columna Vertebral con Resorte Amortiguado	35
3.5	Adaptación de la Columna Vertebral	36
3.6	Animación de la Cola	36
3.7	Coordinación entre patas: Magnet Points	36
3.8	Patrón de Caminata Cuadrúpeda	37
3.9	Secuencia de Movimiento de Patas	37
3.10	Interpolación Parabólica de Patas	37
3.11	Sistema de Cohesión de Patas	38
3.12	Sistema de Salto Adaptativo	38
3.13	Detección de Obstáculos	39
3.14	Cálculo de Trayectoria de Salto	40
3.15	Preparación y Ejecución de Salto	40
3.16	Detección y Adaptación al Terreno	40
3.17	Raycast para Altura de Suelo	41
3.18	Adaptación de Altura de Patas	41
3.19	Validación de Próximos Pasos	42
3.20	Estados de Caminata (PASO_1 a PASO_5)	42
3.21	Estados de Salto (PREP, SALTO, ATERRIZAJE)	43
3.22	Estados de Carrera	44
3.23	Lógica de Transiciones	44
4	Sistema de Animación de Cola	46
4.1	Comportamiento Real	46
4.2	Representación Matemática	47
4.3	Sistema de Personalidad	47
4.3.1	La personalidad como multiplicador	47
4.3.2	Estados emocionales	48
4.4	Movimiento de la cola	48
4.4.1	Ondulación lateral	48
4.4.2	Movimiento tridimensional	49
4.4.3	Reacción al Movimiento del Cuerpo	49
4.4.4	El patrón <i>Observer</i>	50
4.4.5	Amortiguación y suavizado	50
4.5	Sistema Temporal	50

4.5.1	El problema del tiempo	50
4.5.2	Acumulador temporal y <i>delta</i>	50
4.6	Representación Física	51
5	Sistema de IK y Esqueleto	52
5.1	Configuración de Esqueleto en Godot	53
5.1.1	Estructura Jerárquica de Huesos	53
5.1.2	Configuración de Nodos IK	53
5.2	El Problema de la Columna Vertebral	54
5.2.1	Limitaciones del IK Tradicional	54
5.2.2	El Algoritmo de Cuerda	54
5.3	Sistema de Amortiguación con Resortes	55
5.3.1	Física de Resortes	55
5.3.2	Integración de Verlet	56
5.4	Cálculo de Orientación de Vértexes	56
5.4.1	Proyección en Plano Horizontal	56
5.4.2	Rotación Vertical	57
5.5	Coordinación de Sistemas IK	57
5.5.1	Actualización de Posiciones Target	57
5.5.2	Sistema de Magnets	57
5.6	Integración con Transformaciones del Cuerpo	58
5.6.1	Posicionamiento y Orientación del Cuerpo	58
5.6.2	Cálculo de Posición de Cabeza	58
5.7	Consideraciones de Rendimiento	59
5.7.1	Optimización de Cálculos	59
5.7.2	Configuración de Frecuencia	59
IV	Generación de Ciudad Procedural	61
6	Sistema de Chunks Espaciales	62
6.1	Mundos Procedurales: Minecraft	62
6.2	Partición Espacial del Mundo	63
6.2.1	Cálculo de Coordenadas de Chunk	63
6.2.2	El Mundo como una Función	64
6.3	Generación de Ciudades	64
6.3.1	El Algoritmo de Generación	64
6.3.2	Hash Determinístico con Semilla	65
6.3.3	Voronoi: Generación de Ciudades	66
6.4	Sistema de Estabilidad de Chunks	67
6.4.1	Algoritmo de Generación	68
6.4.2	Sistema de Caché Básico	69
7	Sistema de generación procedural de casas	70
7.1	Bloque I: Construcción geométrica de una casa	71
7.1.1	Representación geométrica abstracta	71
7.1.2	Etapas de construcción	71

7.1.3	Incorporación de aleros	71
7.1.4	Lógica para la creación de buhardillas	73
7.1.5	Incorporación de ventanas	74
7.2	Bloque II: Representación en Godot	75
7.2.1	Conversión de geometría a malla	75
7.2.2	Texturizado y proyección UV	75
7.2.3	Cálculo de tangentes	75
7.2.4	Separación por materiales	76
7.2.5	Creación del nodo visual final	76

V Sistema de Interfaz y Visualización 77

8 Sistema de Cámara Dinámico 78

8.1	El Problema de la Cámara 3D	78
8.2	Arquitectura de Cámara Orbital	79
8.2.1	Coordenadas Esféricas	79
8.2.2	Restricciones de Movimiento	79
8.3	Suavizado de Movimiento	80
8.3.1	Suavizado de Posición	80
8.3.2	Suavizado de Rotación con Cuaterniones	80
8.4	Control de Zoom Inteligente	80
8.4.1	Zoom Continuo vs Discreto	80

9 Sistema de Efectos Visuales 82

9.1	El Problema de la Retroalimentación Visual	82
9.1.1	Raycast 3D desde Input 2D	82
9.2	Explosiones de Click: Diseño Orientado a Superficies	82
9.2.1	Cálculo de Base Ortonormal	83
9.2.2	Geometría de Explosión: Torus Expansivo	83
9.3	Sistema de Partículas Físicas	83
9.3.1	Distribución Espacial Realista	84
9.3.2	Simulación Física Simplificada	84

Parte I

Introduccion

0.1. Descripción del Proyecto

Este documento presenta la implementación completa de una simulación de gato con comportamiento procedural realista en Godot 4, junto con un sistema de generación procedural de ciudades. El proyecto combina técnicas avanzadas de animación procedural, física de comportamiento animal, y generación geométrica para crear una experiencia interactiva donde un gato puede moverse de manera natural por entornos urbanos generados dinámicamente.

0.2. Objetivos del Documento

Este documento tiene como objetivo principal permitir que **cualquier persona** pueda reproducir completamente el proyecto sin necesidad de conocimientos adicionales o búsquedas en internet. Al finalizar la lectura, el lector será capaz de:

- Implementar un sistema completo de locomoción cuadrúpeda procedural
- Crear un generador de ciudades usando diagramas de Voronoi y triangulación de Delaunay
- Desarrollar sistemas de animación procedural para comportamiento animal
- Manejar sistemas de chunks espaciales para mundos grandes
- Implementar algoritmos de geometría computacional en Godot

0.3. Conocimientos Previos Asumidos

Se asume que el lector tiene conocimientos básicos en:

- **Programación general:** Variables, funciones, condicionales, bucles, clases
- **Matemáticas básicas:** Álgebra elemental, trigonometría básica
- **Conceptos de computación gráfica básicos:** Qué es un vértice, una textura, un polígono, mapas uv.
- **Instalación de software:** Capacidad de descargar e instalar Godot 4

0.4. Conocimientos NO Asumidos

No se requiere conocimiento previo en:

- Godot Engine (se explicará desde cero)
- GDScript (se enseñará completamente)
- Matemáticas 3D avanzadas (se explicarán todos los conceptos)

- Geometría computacional (se desarrollará paso a paso)
- Animación procedural (se construirá desde los fundamentos)
- Física de videojuegos (se explicará lo necesario)

0.5. Estructura del Documento

El documento está organizado siguiendo un árbol de dependencias estricto. Cada capítulo depende únicamente de los capítulos anteriores, garantizando una progresión lógica del aprendizaje.

Orden de Lectura Obligatorio

Es FUNDAMENTAL leer los capítulos en orden secuencial. Cada capítulo utiliza conceptos, técnicas y código desarrollados en los capítulos anteriores.

0.5.1. Arquitectura de Dependencias

El documento fue escrito siguiendo este orden. Si gusta puede saltarse los capítulos que explican conceptos básicos.

1. **Fundamentos (Capítulos 2-5):** Bases de Godot y matemáticas
2. **Sistemas del Gato (Capítulos 6-9):** Comportamiento y animación
3. **Generación Procedural (Capítulos 10-12):** Ciudad y terreno
4. **Interfaz (Capítulos 13-15):** Cámara y efectos visuales

0.6. Tiempo Estimado

- **Lectura completa:** 8-12 horas
- **Implementación siguiendo el documento:** 25-72 horas
- **Tiempo total del proyecto:** 25-35 horas

Parte II

Fundamentos

Capítulo 1

Fundamentos de Godot 4

1.1. Instalación y Configuración

Godot 4 representa una evolución significativa en el mundo de los motores de juegos gratuitos y de código abierto. A diferencia de otros motores que requieren configuraciones complejas o licencias costosas, Godot te permite comenzar a desarrollar inmediatamente después de la descarga.

La instalación es sorprendentemente directa. Simplemente descargas el ejecutable desde el sitio oficial de Godot, lo extraes y ya puedes ejecutarlo. No hay instaladores complicados, registros obligatorios o configuraciones iniciales tediosas. Esta simplicidad es una de las características más atractivas del motor.



Figura 1.1: Pagina oficial Godot

Al abrir Godot por primera vez, te encuentras con el Project Manager, que es tu centro de control para todos tus proyectos. Aquí puedes crear nuevos proyectos, importar existentes o abrir proyectos recientes. Cuando creas un nuevo proyecto, Godot te pregunta por la ubicación, el nombre y el tipo de renderizado que deseas usar.

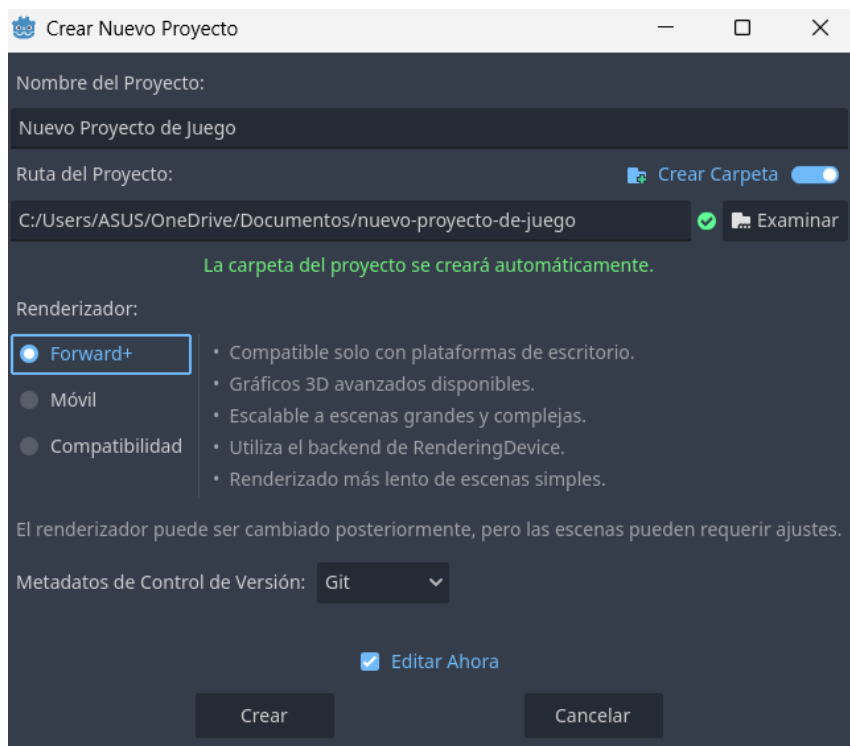


Figura 1.2: Centro de control Godot

La configuración del proyecto es donde defines las características fundamentales de tu juego. En la configuración de proyecto puedes establecer la resolución base, configurar las capas de física, definir grupos de nodos, y ajustar múltiples parámetros que afectarán todo tu desarrollo. Es importante entender que estas configuraciones son globales y afectan todo el comportamiento del proyecto.

Un aspecto crucial de la configuración inicial es entender los diferentes tipos de renderizado que ofrece Godot 4. Tienes tres opciones principales: Forward+, Mobile y Compatibility. Forward+ es el más moderno y potente, ideal para juegos de PC y consolas. Mobile está optimizado para dispositivos móviles, mientras que Compatibility asegura que tu juego funcione en hardware más antiguo.

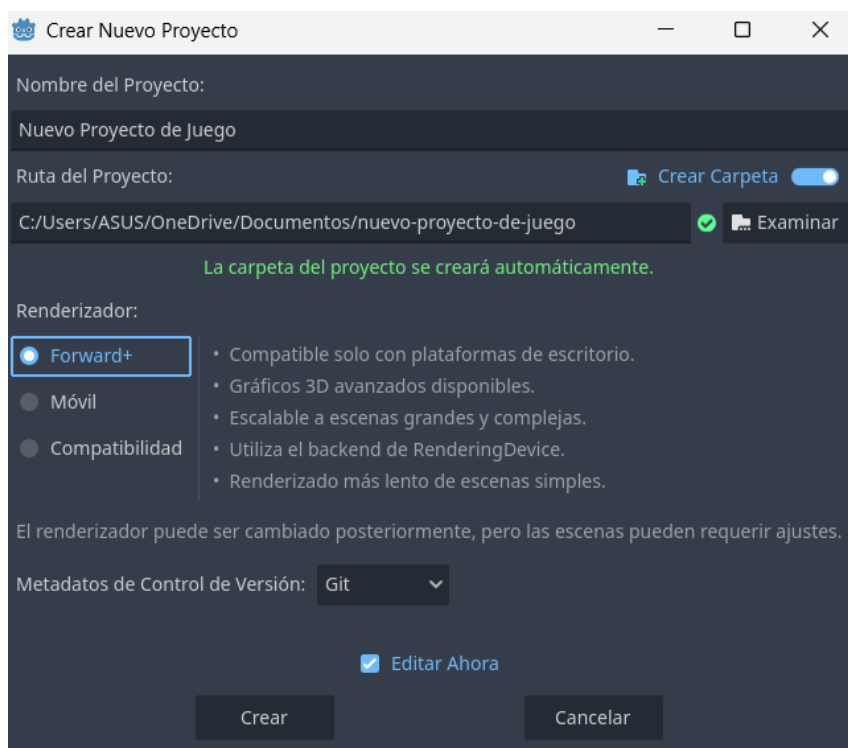


Figura 1.3: Creación de proyecto en Godot

1.2. Estructura de Escenas y Nodos

El concepto de escenas y nodos es el corazón de Godot y lo que lo distingue de otros motores. Imagina que estás construyendo con bloques de LEGO, donde cada bloque tiene una función específica y puedes combinarlos para crear estructuras más complejas.

Un nodo es la unidad básica de construcción en Godot. Cada nodo tiene una funcionalidad específica: algunos manejan gráficos 2D, otros física 3D, algunos reproducen audio, otros gestionan la entrada del usuario. La belleza del sistema está en cómo puedes combinar estos nodos para crear funcionalidades complejas.

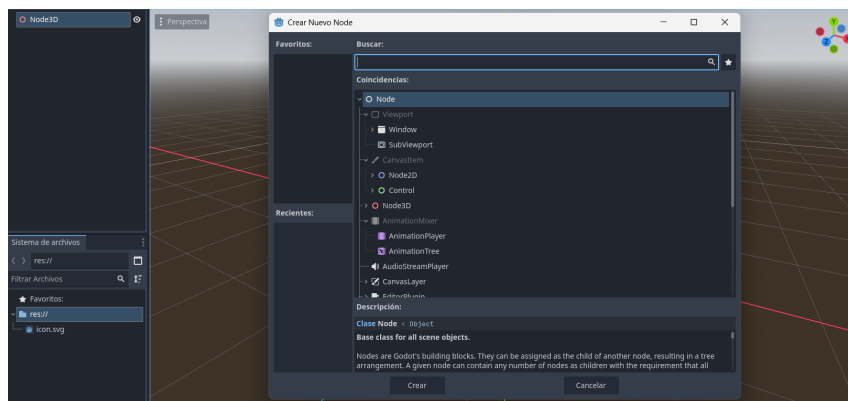


Figura 1.4: Creación nodo hijo en Godot

Una escena es una colección de nodos organizados en una estructura jerárquica de árbol. Cada escena tiene un nodo raíz y puede contener múltiples nodos hijos. Esta estructura de árbol define las relaciones padre-hijo entre los nodos, lo que afecta cómo se comportan en términos de transformaciones, visibilidad y procesamiento.

La jerarquía es fundamental para entender cómo funciona Godot. Cuando un nodo padre se mueve, todos sus hijos se mueven con él. Si un nodo padre se oculta, todos sus hijos también se ocultan. Esta relación jerárquica permite crear sistemas complejos de forma intuitiva.

Los tipos de nodos están diseñados para propósitos específicos. Un `Node2D` maneja posicionamiento y transformaciones en 2D, un `RigidBody2D` gestiona física de cuerpos rígidos, un `AudioStreamPlayer` reproduce sonidos, y así sucesivamente. Cada tipo de nodo viene con propiedades y métodos específicos para su función.

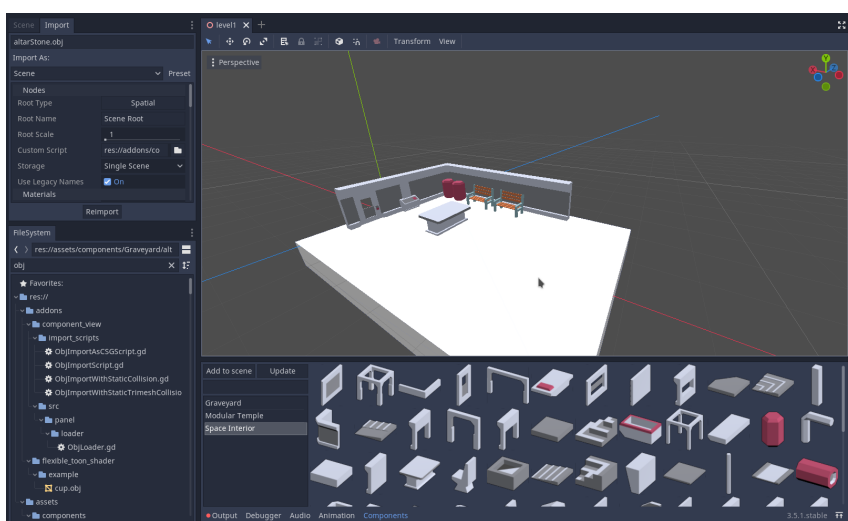


Figura 1.5: Ejemplo de escena en Godot

1.2.1. Herencia e Instanciación

La herencia en Godot funciona de manera similar a la programación orientada a objetos, pero aplicada a escenas completas. Cuando creas una escena, puedes guardarla como un archivo `.tscn` y luego instanciarla en otras escenas. Esto significa que puedes crear una escena de `Enemigo` y luego usar múltiples instancias de esa escena en tu nivel principal.

La herencia permite modificar instancias específicas sin afectar la escena original. Por ejemplo, puedes tener una escena base de "Personajes" crear variantes como "Guerrero" o "Mago" que heredan las características básicas pero añaden funcionalidades específicas.

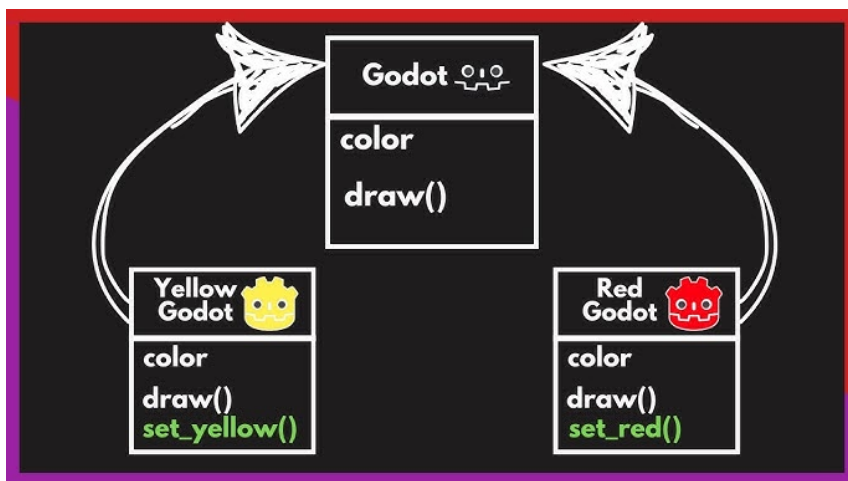


Figura 1.6: Herencia en Godot

Cuando instancias una escena, Godot crea una copia independiente de toda la estructura de nodos. Cada instancia mantiene su propio estado y puede ser modificada individualmente. Sin embargo, si cambias la escena original, todos los aspectos no modificados en las instancias se actualizarán automáticamente.

El sistema de herencia también permite sobrescribir propiedades y comportamientos en instancias específicas. Esto significa que puedes tener un enemigo base y crear variaciones cambiando solo las propiedades necesarias, como la velocidad, el color o el tamaño, sin duplicar todo el código.

1.2.2. Carga Dinámica de Escenas

La carga dinámica de escenas es una técnica poderosa que permite cargar y descargar contenido durante la ejecución del juego. Esto es esencial para manejar la memoria eficientemente y crear experiencias de juego fluidas.

Godot proporciona varias formas de cargar escenas dinámicamente. La más común es usar `load()` para cargar un recurso de escena y luego `instantiate()` para crear una instancia de esa escena. También puedes usar `preload()` cuando quieres cargar la escena en tiempo de compilación, lo que es más eficiente si sabes que vas a usar la escena con certeza.

La carga dinámica es especialmente útil para sistemas como inventarios, donde puedes cargar diferentes tipos de objetos según sea necesario, o para sistemas de niveles donde cargas el siguiente nivel solo cuando el jugador está listo para avanzar.

Un aspecto importante de la carga dinámica es la gestión de memoria. Cuando cargas escenas dinámicamente, debes asegurarte de liberarlas cuando ya no las necesitas usando `queue_free()` o `free()`. Esto previene fugas de memoria que pueden afectar el rendimiento del juego.

1.3. Programación en GDScript

GDScript es el lenguaje de programación nativo de Godot, diseñado específicamente para el desarrollo de juegos. Su sintaxis está inspirada en Python, lo que

lo hace accesible para principiantes pero lo suficientemente potente para proyectos complejos.

Una de las ventajas principales de GDScript es su integración perfecta con el editor de Godot. Tienes autocompletado inteligente, depuración integrada, y el lenguaje está optimizado para las operaciones comunes en el desarrollo de juegos. Además, al estar diseñado específicamente para Godot, el código tiende a ser más conciso y legible que en otros lenguajes.

GDScript es tipado dinámicamente, pero también soporta tipado estático opcional. Esto significa que puedes escribir código rápidamente sin preocuparte por los tipos, pero también puedes añadir tipos cuando quieras más control y mejor rendimiento. El compilador de GDScript puede optimizar mejor el código cuando conoce los tipos de las variables.



```

148
149 func patron_cola_relajada() -> Vector3:
150     # Movimiento suave en forma de "S", como gato descansando pero atento
151     var vector_lateral = Vector3(direccion.z, 0, -direccion.x).normalized()
152     var vector_atras = -direccion.normalized()
153
154     var lateral = (sin(cola_tiempo * 0.8) * 0.4 + cola_direccion_preferida * 0.3) * AMPLITUD_LATERAL
155     var y = ALTURA_BASE Cola + sin(cola_tiempo * 0.5) * AMPLITUD_FLOTACION * 0.3
156     var atras = LONGITUD_Cola * 0.8 + sin(cola_tiempo * 0.6) * LONGITUD_Cola * 0.1
157
158     return vector_lateral * lateral + Vector3(0, y, 0) + vector_atras * atras
159
160 func patron_cola_curiosa() -> Vector3:
161     # Movimiento más animado, como gato explorando
162     # Combina ondulación lateral con movimiento vertical más pronunciado
163     var factor_velocidad = min(VELOCIDAD_BASE, 2.0)
164     var vector_lateral = Vector3(direccion.z, 0, -direccion.x).normalized()
165     var vector_atras = -direccion.normalized()
166
167     var lateral = (sin(cola_tiempo * 1.2) + sin(cola_tiempo * 2.1) * 0.3) * 0.6 * AMPLITUD_LATERAL
168     # Agregar "capricho" extra cuando está curioso
169     lateral += sin(cola_tiempo * 3.2) * 0.2 * FACTOR_PERSONALIDAD * AMPLITUD_LATERAL
170
171     var y = ALTURA_BASE Cola + sin(cola_tiempo * 1.0) * AMPLITUD_FLOTACION * 0.6 * factor_velocidad
172     var atras = LONGITUD_Cola * 0.6 + cos(cola_tiempo * 0.8) * LONGITUD_Cola * 0.2

```

Figura 1.7: Ejemplo GDScript

La estructura básica de un script en GDScript incluye la declaración de la clase (con **extends**), la declaración de variables, y la definición de funciones. El sistema de herencia permite que tu script herede todas las funcionalidades del nodo al que está adjunto, plus cualquier funcionalidad adicional que definas.

1.3.1. Variables Exportadas

Las variables exportadas son una característica poderosa que permite exponer variables de tu script en el editor de Godot. Cuando marcas una variable como exportada usando **@export**, aparece en el inspector del nodo, permitiendo que diseñadores y otros desarrolladores ajusten valores sin tocar el código.

Esta funcionalidad es especialmente valiosa para crear scripts reutilizables y configurables. Por ejemplo, puedes crear un script de movimiento de enemigo donde la velocidad, el rango de patrullaje y el tiempo de espera sean variables exportadas. Esto permite usar el mismo script para diferentes tipos de enemigos con comportamientos variados.

```
6  @export var VELOCIDAD_BASE_INICIAL = 1.2
7  @export var VELOCIDAD_BASE = VELOCIDAD_BASE_INICIAL
8  @export var VELOCIDAD_MOVIMIENTO = 0.7
9  @export var VELOCIDAD_ROTACION = 0.1
10 @export var DISTANCIA_ENTRE_PATAS = 12 * ESCALA
11 @export var LONGITUD_PASO = 220 * ESCALA
12 @export var UMBRAL_DISTANCIA = 200 * ESCALA
13 @export var UMBRAL_PATAS_FRONT = 140 * ESCALA
14 @export var UMBRAL_PATAS_BACK = 110 * ESCALA
15 @export var ALTURA_PASO = 70 * ESCALA
16 @export var RANDOM_DISTANCE = 0.0
17 @export var MIN_HEIGHT_JUMP = 120 * ESCALA
18 @export var MAX_HEIGHT_JUMP = 120 * 5 * ESCALA
19 @export var UMBRAL_DETECCION_SALTO = 270 * ESCALA
20 @export var MARGEN_SALTO_ADELANTE = 50 * ESCALA
21 @export var MOVEMENT_TARGET_SPEED = 1.5
22 @export var DEBUG = false
23
```

Figura 1.8: Ejemplo de variables exportadas

Las variables exportadas soportan diferentes tipos de datos: números, strings, booleanos, vectores, colores, recursos, y más. Godot automáticamente genera la interfaz apropiada en el inspector según el tipo de variable. Para un booleano obtienes un checkbox, para un color obtienes un picker de color, para un recurso obtienes un campo de arrastrar y soltar.

También puedes añadir metadatos a las variables exportadas para controlar cómo aparecen en el inspector. Puedes establecer rangos mínimos y máximos para números, proporcionar listas de valores válidos para enums, o incluso crear interfaces personalizadas para tipos de datos complejos.

1.3.2. Señales y Métodos

Las señales son el sistema de comunicación entre nodos en Godot, similar al patrón observer en programación. Una señal es un mensaje que un nodo puede enviar cuando algo importante sucede, y otros nodos pueden escuchar estas señales y reaccionar apropiadamente.

El sistema de señales promueve un acoplamiento débil entre los componentes de tu juego. En lugar de que un nodo tenga referencias directas a otros nodos, puede simplemente emitir una señal cuando algo sucede. Los nodos interesados pueden conectarse a esta señal y responder sin que el emisor necesite conocer quién está escuchando.

Para crear una señal, la declaras en tu script usando `signal nombre_de_señal`. Luego puedes emitir la señal en cualquier momento usando `emit_signal("nombre_de_señal")`. Otros nodos pueden conectarse a esta señal usando `connect()` o a través del editor de Godot.

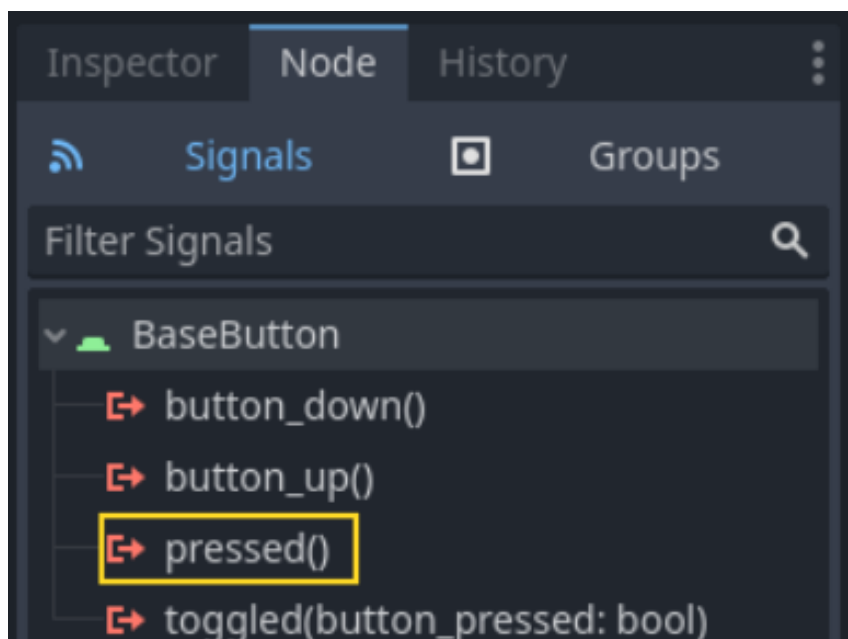


Figura 1.9: Creación de signals en Godot

Las señales pueden llevar parámetros, permitiendo que el emisor proporcione información adicional sobre el evento. Por ejemplo, una señal de "jugador_dañado" podría incluir la cantidad de daño como parámetro. Los métodos que se conectan a la señal deben tener la misma cantidad y tipos de parámetros.

Los métodos en GDScript siguen las convenciones estándar de funciones. Puedes definir parámetros opcionales con valores por defecto, usar parámetros nombrados al llamar funciones, y retornar valores múltiples usando arrays o diccionarios. El sistema de métodos también soporta sobrecarga básica a través del uso de parámetros opcionales.

1.3.3. Ciclo de Vida (`_ready`, `_process`)

El ciclo de vida de un nodo en Godot está definido por una serie de métodos especiales que se llaman automáticamente en momentos específicos. Los dos más importantes son `_ready()` y `_process()`, que forman la base de la mayoría de la lógica de juego.

```
func _ready():  
    >| setup_core_systems()  
    >| setup_debug_overlay()  
    >| find_player_reference()  
    >| call_deferred("generate_initial_city")
```

Figura 1.10: Ejemplo de función `_ready()`

El método `_ready()` se ejecuta una sola vez cuando el nodo entra en el árbol de escenas y todos sus hijos están listos. Este es el lugar ideal para realizar inicializaciones, configurar referencias a otros nodos, conectar señales, y preparar el estado inicial del nodo. Es importante entender que `_ready()` se llama en orden desde las hojas del árbol hacia la raíz, garantizando que todos los nodos hijos estén listos antes que sus padres.

```
▼ func _process(_delta):  
    >I update_player_position()  
    >I handle_debug_input()  
    >I debug_control.queue_redraw()  
    >I  
    ▼ >I if should_regenerate_city():  
        >I >I call_deferred("regenerate_city")
```

Figura 1.11: Ejemplo de función `_process()`

El método `_process(delta)` se ejecuta una vez por frame durante toda la vida del nodo. Recibe como parámetro `delta`, que representa el tiempo transcurrido desde el último frame en segundos. Este método es donde colocas la lógica que debe ejecutarse continuamente, como movimiento, animaciones, verificaciones de estado, y actualizaciones de interfaz.

Existe también `_physics_process(delta)`, que funciona similar a `_process()` pero se ejecuta a una frecuencia fija (por defecto 60 FPS) y está sincronizado con el sistema de física. Usa este método para lógica relacionada con física, como movimiento de personajes controlados por el jugador o detección de colisiones críticas.

El parámetro `delta` es crucial para crear movimiento independiente del frame-rate. En lugar de mover un objeto una cantidad fija cada frame, lo mueves por `velocidad * delta`. Esto asegura que el movimiento sea consistente sin importar si el juego corre a 30, 60 o 120 FPS.

Otros métodos importantes del ciclo de vida incluyen `_enter_tree()` (cuando el nodo entra al árbol), `_exit_tree()` (cuando sale del árbol), y `_input()` (para manejar entrada del usuario). Cada uno tiene su propósito específico y entender cuándo usar cada uno es fundamental para crear código eficiente y sin errores.

1.4. Sistema de Materiales

El sistema de materiales en Godot 4 es donde la magia visual realmente cobra vida. Un material define cómo se ve la superficie de un objeto: su color, textura, brillo, transparencia, y cómo interactúa con la luz. Piensa en los materiales como las "pieles" que vistes a tus modelos 3D para darles personalidad y realismo.

Godot 4 introduce un sistema de materiales completamente renovado que aprovecha las capacidades modernas de renderizado. El motor soporta múltiples tipos de materiales, pero los más importantes son los materiales PBR (Physically Based Rendering) que simulan cómo se comporta la luz en el mundo real.



Figura 1.12: PBR materials aplicados a un objeto en Godot

El concepto fundamental detrás de los materiales PBR es que la luz se comporta de manera predecible y matemáticamente consistente. En lugar de ajustar valores artísticos arbitrarios, trabajas con propiedades físicas reales como metalicidad, rugosidad, y reflectancia. Esto significa que tus materiales se verán consistentes bajo diferentes condiciones de iluminación y serán más fáciles de crear una vez que entiendas los principios básicos.

El flujo de trabajo típico para crear materiales involucra combinar diferentes tipos de texturas que controlan aspectos específicos del material. Una textura albedo define el color base, una textura normal simula detalles de superficie, una textura de rugosidad controla qué tan lisa o áspera es la superficie, y así sucesivamente.

1.4.1. Creación de Materiales PBR

Los materiales PBR (Physically Based Rendering) representan el estándar moderno para renderizado realista. En lugar de trabajar con parámetros artísticos arbitrarios, los materiales PBR se basan en propiedades físicas reales que puedes medir en el mundo real.

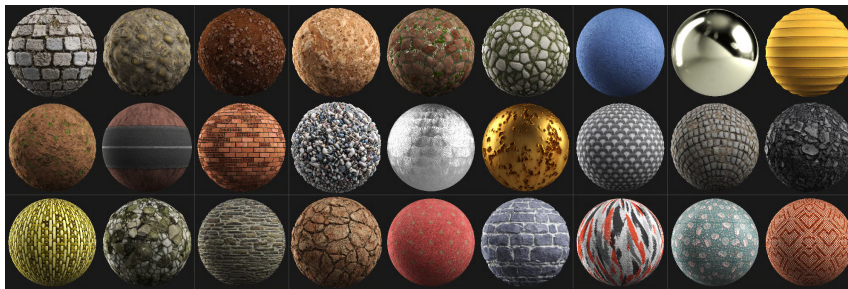


Figura 1.13: Ejemplo de materiales PBR

Para crear un material PBR en Godot, comienzas con un `StandardMaterial3D` y ajustas sus propiedades principales. El albedo define el color base del material, como si fuera el color que verías bajo luz blanca perfecta. Es importante entender que el albedo no incluye información de iluminación, solo el color intrínseco del material.

La metalicidad (**metallic**) es un valor binario que define si el material es metálico o no metálico. Los materiales metálicos reflejan la luz de manera diferente a los no metálicos. Los metales no tienen color difuso y toda su apariencia viene de las reflexiones, mientras que los no metales tienen tanto color difuso como reflexiones.

La rugosidad (**roughness**) controla qué tan lisa o áspera es la superficie del material. Una superficie completamente lisa (rugosidad 0) produce reflexiones perfectas como un espejo, mientras que una superficie muy rugosa (rugosidad 1) dispersa la luz en todas las direcciones, creando un aspecto mate.

El mapa de normales (normal map) es una textura especial que simula detalles de superficie sin añadir geometría real. Funciona perturbando las normales de la superficie para que la luz se comporte como si hubiera detalles microscópicos presentes. Esto permite crear superficies con apariencia compleja manteniendo la geometría simple.

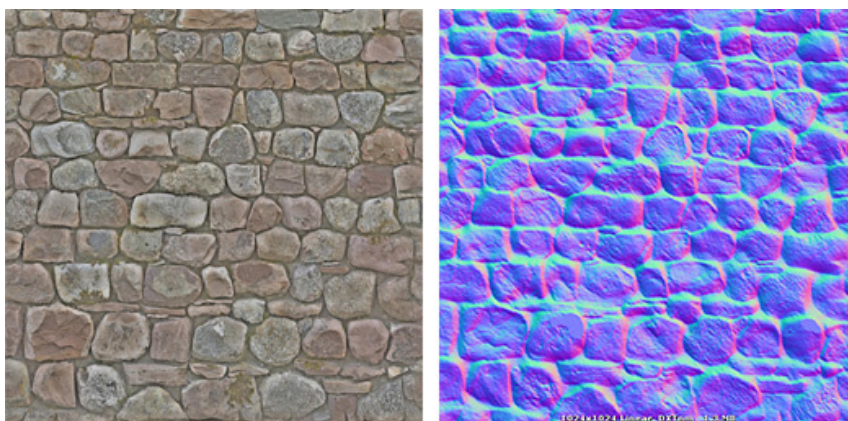


Figura 1.14: Mapa de normales

La emisión (**emission**) permite que el material emita luz propia. Esto es útil para crear materiales que brillan, como pantallas, luces LED, o materiales mágicos. La emisión no afecta otros objetos por defecto, pero puedes configurar el material para que contribuya a la iluminación global.

1.4.2. Mapeo UV y Texturas

El mapeo UV es el proceso de proyectar texturas 2D sobre superficies 3D. Imagina que estás envolviendo un regalo: necesitas decidir cómo el papel de regalo 2D se va a adaptar a la forma 3D del objeto. El mapeo UV hace exactamente eso con las texturas.

Las coordenadas UV representan posiciones en el espacio 2D de la textura. La U es horizontal (como X) y la V es vertical (como Y). Cada vértice de tu modelo 3D tiene asignadas coordenadas UV que determinan qué parte de la textura se aplicará a ese punto.

Un buen mapeo UV es crucial para que las texturas se vean correctas. Debes minimizar la distorsión (evitar que las texturas se estiren o compriman), maximizar el uso del espacio UV (no desperdiciar píxeles de textura), y mantener la coherencia de escala (objetos similares deben tener densidad de textura similar).

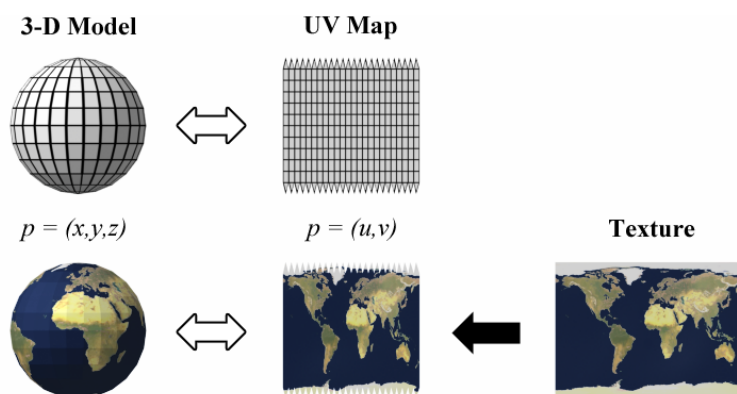


Figura 1.15: Mapeo de coordenadas UV

Godot proporciona herramientas básicas para trabajar con UVs, pero para mapeos complejos generalmente necesitarás usar software de modelado 3D como Blender. El flujo de trabajo típico es crear el modelo, desenvolverlo (unwrap) para generar las coordenadas UV, y luego crear las texturas basándose en este desenvolvimiento.

Las texturas en Godot pueden ser de varios tipos: albedo para color base, normal para detalles de superficie, roughness para rugosidad, metallic para propiedades metálicas, emission para emisión, y más. Cada tipo de textura debe estar en el formato correcto y tener las propiedades adecuadas para funcionar correctamente con el sistema de materiales.

Es importante entender la resolución de texturas y cómo afecta el rendimiento. Texturas más grandes proporcionan más detalle pero consumen más memoria y ancho de banda. Debes balancear la calidad visual con el rendimiento, especialmente en dispositivos móviles donde los recursos son limitados.

1.5. Sistema de Física de Godot

El sistema de física en Godot 4 es robusto y versátil, diseñado para manejar todo desde simulaciones simples hasta interacciones físicas complejas. Godot utiliza su propio motor de física integrado, lo que significa que tienes control total sobre cómo se comporta la física en tu juego.

La física en Godot se basa en el concepto de cuerpos (**bodies**) que existen en un espacio físico y pueden interactuar entre sí. Cada cuerpo tiene propiedades como masa, fricción, rebote, y forma de colisión. El motor de física calcula las fuerzas, velocidades, y colisiones automáticamente, pero tú tienes control sobre cómo los objetos responden a estas interacciones.

Una característica importante del sistema de física de Godot es que está integrado con el sistema de nodos. Los cuerpos físicos son nodos que puedes colocar en tu escena, configurar en el editor, y controlar con scripts. Esta integración hace que trabajar con física sea intuitivo y accesible.

El sistema soporta tanto física 2D como 3D, cada una con sus propios tipos de nodos y características. Aunque los principios fundamentales son similares, hay diferencias importantes en cómo se implementan las colisiones, las fuerzas, y las restricciones en cada dimensión.

1.5.1. Cuerpos Rígidos (StaticBody3D)

Los cuerpos estáticos (**StaticBody3D**) son los elementos más básicos del sistema de física. Como su nombre indica, estos cuerpos no se mueven por fuerzas físicas, pero pueden detectar colisiones con otros objetos y proporcionar superficies sólidas contra las que otros cuerpos pueden colisionar.

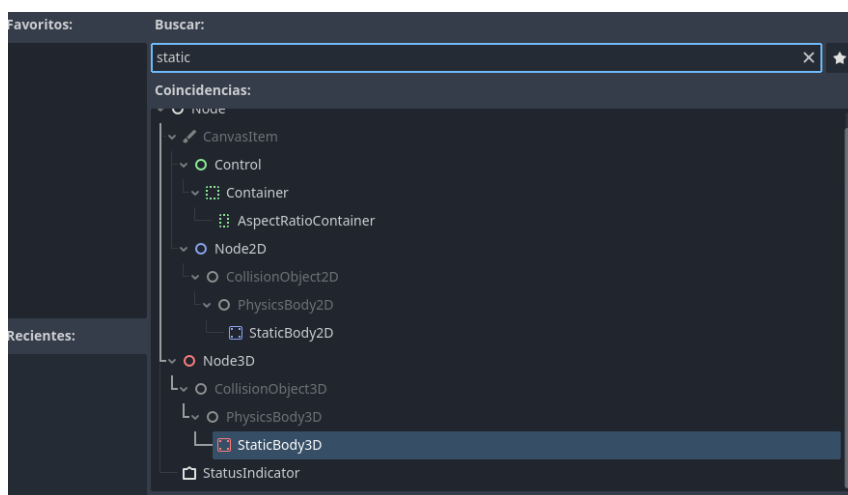


Figura 1.16: Creación de StaticBody3D

Un **StaticBody3D** es perfecto para crear el entorno estático de tu juego: paredes, suelos, techos, plataformas, y cualquier elemento arquitectónico que no necesite moverse. Aunque no se mueven por física, puedes moverlos programáticamente cam-

biando su posición, lo que es útil para crear plataformas móviles o elementos de nivel que se activan por eventos.

Para que un **StaticBody3D** funcione correctamente, necesita un **CollisionShape3D** como hijo. El **CollisionShape3D** define la forma invisible que el motor de física usa para detectar colisiones. Esta forma puede ser simple (como una caja o esfera) o compleja (como una malla triangular detallada).

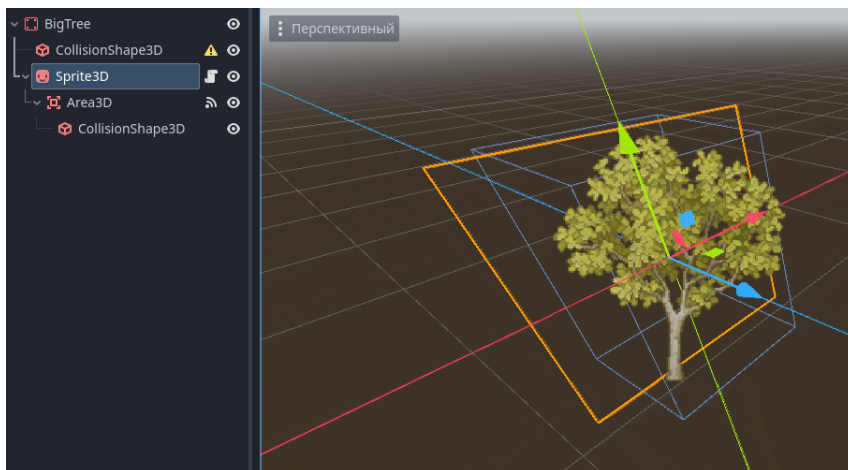


Figura 1.17: Definición de colisiones en un objeto

Es importante entender la diferencia entre la forma visual de un objeto y su forma de colisión. La forma visual es lo que ve el jugador, mientras que la forma de colisión es lo que usa el motor de física. Para optimizar el rendimiento, generalmente usas formas de colisión más simples que las formas visuales.

Los **StaticBody3D** tienen propiedades como fricción y rebote que afectan cómo otros objetos interactúan con ellos. Una superficie con alta fricción hace que los objetos se deslicen menos, mientras que una superficie con alto rebote hace que los objetos reboten más energéticamente.

1.5.2. Sistema de Raycast

El raycast es una herramienta fundamental en el desarrollo de juegos que permite disparar un rayo invisible desde un punto en una dirección específica y detectar qué objetos intercepta. Es como tener un láser invisible que puede decirte qué hay en su camino.

En Godot, el raycast se implementa principalmente a través del nodo **RayCast3D**. Este nodo dispara un rayo desde su posición en la dirección que especifiques y puede detectar colisiones con objetos físicos. Es increíblemente útil para una amplia variedad de aplicaciones en juegos.

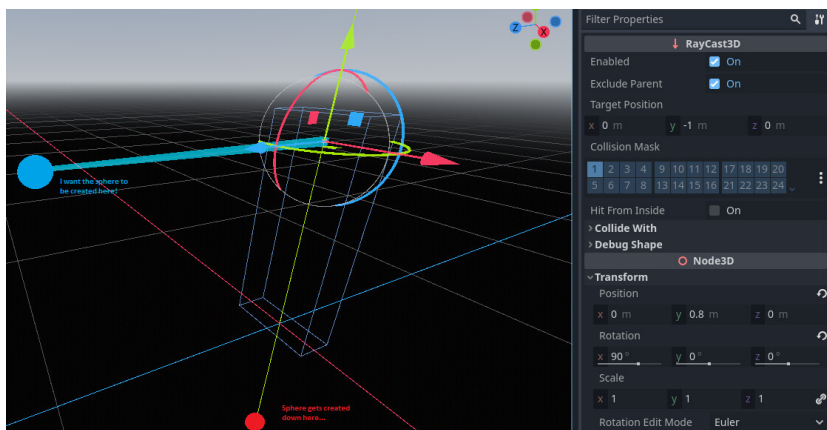


Figura 1.18: Configuración de RayCast3D

Una aplicación común del raycast es la detección de línea de visión. Puedes usar un rayo para determinar si un enemigo puede ver al jugador disparando un rayo desde el enemigo hacia el jugador. Si el rayo no es bloqueado por obstáculos, el enemigo tiene línea de visión clara.

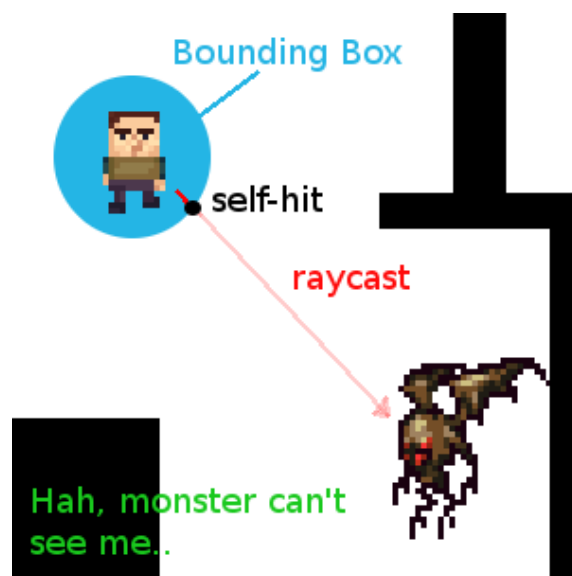


Figura 1.19: Explicación visual de conceptos de Raycast

El raycast también es esencial para implementar sistemas de disparo realistas. Cuando el jugador dispara un arma, puedes usar un rayo para determinar exactamente qué objeto golpea la bala y dónde. Esto es más preciso y eficiente que usar proyectiles físicos reales para cada disparo.

Para detección de suelo, el raycast es invaluable. Puedes disparar un rayo hacia abajo desde un personaje para determinar si está sobre suelo sólido, cuál es la distancia al suelo, o qué tipo de superficie está pisando. Esto es crucial para implementar saltos, animaciones de aterrizaje, y efectos de partículas apropiados.

El `RayCast3D` te permite configurar varios parámetros: la distancia máxima del rayo, qué capas de colisión debe considerar, si debe detectar áreas además de cuerpos físicos, y si debe excluir el cuerpo padre del rayo.

1.5.3. Detección de Colisiones

La detección de colisiones es el corazón del sistema de física, determinando cuándo y cómo los objetos interactúan entre sí. Godot proporciona múltiples métodos para detectar colisiones, cada uno optimizado para diferentes tipos de interacciones.

El método más directo es usar las señales que emiten los cuerpos físicos cuando ocurren colisiones. Por ejemplo, `RigidBody3D` emite señales como `body_entered` cuando otro cuerpo entra en contacto con él. Estas señales te permiten responder inmediatamente a las colisiones sin necesidad de verificar constantemente el estado de colisión.

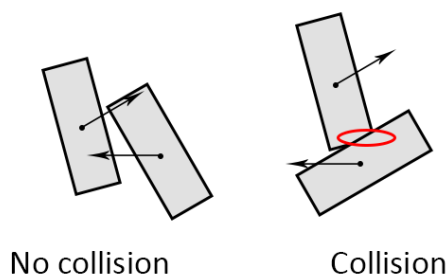


Figura 1.20: Colision ejemplo

Para detección más granular, puedes consultar directamente el estado de colisión de los cuerpos. Los métodos como `get_colliding_bodies()` te dan una lista de todos los cuerpos que están actualmente colisionando con el objeto. Esto es útil cuando necesitas información detallada sobre múltiples colisiones simultáneas.

Las capas de colisión son una herramienta poderosa para organizar qué objetos pueden colisionar entre sí. Puedes asignar diferentes objetos a diferentes capas y configurar qué capas interactúan entre sí. Por ejemplo, puedes tener una capa para jugadores, otra para enemigos, y otra para proyectiles, y configurar que los proyectiles solo colisionen con enemigos.

Los grupos de colisión van un paso más allá y te permiten crear categorías lógicas de objetos. Puedes añadir nodos a grupos y luego verificar si un objeto colisionante pertenece a un grupo específico. Esto es especialmente útil para crear sistemas de daño, recolección de objetos, y activación de triggers.

La detección de colisiones también incluye información detallada sobre el punto de contacto, la normal de la superficie, y la velocidad relativa. Esta información es valiosa para crear efectos realistas como partículas de impacto, sonidos de colisión apropiados, y cálculos de daño basados en la velocidad del impacto.

Es importante optimizar la detección de colisiones para mantener un buen rendimiento. Usar formas de colisión simples cuando sea posible, limitar el número de verificaciones de colisión por frame, y usar las capas de colisión eficientemente puede hacer una gran diferencia en el rendimiento de tu juego.

Capítulo 2

Matemáticas 3D Básicas

2.1. Operaciones con Vector3

Los vectores tridimensionales son la columna vertebral de cualquier aplicación 3D. En Godot, el tipo `Vector3` representa un punto en el espacio tridimensional o una dirección con magnitud. Puedes pensar en un vector como una flecha que tiene tanto dirección como longitud, partiendo desde el origen del sistema de coordenadas.

Un `Vector3` en Godot contiene tres componentes: `x`, `y`, y `z`. Estos representan las coordenadas en los tres ejes del espacio tridimensional. La belleza de trabajar con vectores es que muchas operaciones complejas en 3D se pueden expresar como operaciones matemáticas relativamente simples entre vectores.

Las operaciones básicas entre vectores incluyen suma, resta, multiplicación por escalar, y varias operaciones más especializadas como el producto escalar y vectorial. Cada una de estas operaciones tiene significados geométricos específicos que son fundamentales para entender cómo se comportan los objetos en el espacio 3D.

Godot proporciona una implementación optimizada de `Vector3` que incluye muchas funciones útiles predefinidas. Esto significa que raramente necesitarás implementar estas operaciones desde cero, pero entender cómo funcionan te ayudará a usarlas efectivamente en tu código.

La suma de vectores representa el desplazamiento combinado. Si tienes un vector que representa la posición de un objeto y otro que representa su velocidad, sumar estos vectores te da la nueva posición del objeto. Esta operación es fundamental para cualquier sistema de movimiento.

La resta de vectores es igualmente importante y se usa para calcular la dirección de un objeto hacia otro. Si restas la posición del jugador de la posición de un enemigo, obtienes un vector que apunta desde el jugador hacia el enemigo.

2.1.1. Producto Escalar y Vectorial

El producto escalar (dot product) es una operación fundamental que toma dos vectores y devuelve un número escalar. Matemáticamente, el producto escalar de dos vectores **a** y **b** se define como:

$$\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}| |\mathbf{b}| \cos(\theta)$$

donde θ es el ángulo entre los dos vectores.

Esta operación tiene interpretaciones geométricas muy útiles. Cuando el producto escalar es positivo, significa que los vectores apuntan en direcciones generalmente similares. Cuando es negativo, apuntan en direcciones opuestas. Cuando es cero, los vectores son perpendiculares.

En Godot, puedes calcular el producto escalar usando el método `dot()`: `vector_a.dot(vector_b)`. Esta operación es extremadamente útil para determinar si un enemigo está mirando hacia el jugador, calcular la iluminación de una superficie, o determinar si dos objetos se están acercando o alejando.

El producto vectorial (cross product) es otra operación fundamental, pero solo está definida en tres dimensiones. Toma two vectores y devuelve un tercer vector que es perpendicular a ambos. La dirección del vector resultante sigue la regla de la mano derecha.

$$\mathbf{a} \times \mathbf{b} = (a_y b_z - a_z b_y, a_z b_x - a_x b_z, a_x b_y - a_y b_x)$$

El producto vectorial es especialmente útil para calcular normales de superficie. Si tienes dos vectores que definen un plano, su producto vectorial te da un vector perpendicular a ese plano, que es exactamente lo que necesitas para calcular iluminación o reflexiones.

En Godot, el producto vectorial se calcula con `vector_a.cross(vector_b)`. La magnitud del vector resultante es igual al área del paralelogramo formado por los dos vectores originales, lo que es útil para cálculos de área y volumen.

2.1.2. Normalización de Vectores

La normalización es el proceso de convertir un vector en un vector unitario (con magnitud 1) que mantiene la misma dirección. Un vector normalizado se llama vector unitario y es extremadamente útil cuando solo te interesa la dirección, no la magnitud.

Para normalizar un vector, divides cada componente por la magnitud del vector:

$$\hat{\mathbf{v}} = \frac{\mathbf{v}}{|\mathbf{v}|}$$

donde $|\mathbf{v}| = \sqrt{v_x^2 + v_y^2 + v_z^2}$ es la magnitud del vector.

En Godot, puedes normalizar un vector usando el método `normalized()`, que devuelve una nueva instancia del vector normalizado, o `normalize()`, que modifica el vector original. Es importante verificar que el vector no sea cero antes de normalizarlo, ya que esto causaría una división por cero.

Los vectores normalizados son fundamentales para muchas operaciones en 3D. Se usan para representar direcciones puras, calcular direcciones de movimiento, definir orientaciones de objetos, y en muchos algoritmos de iluminación y sombreado.

Una aplicación común de la normalización es en el movimiento de personajes. Si quieres que un personaje se mueva hacia un punto específico, calculas la dirección (restando posiciones), normalizas el vector resultante, y luego lo multiplicas por la velocidad deseada.

2.1.3. Cálculo de Distancias y Ángulos

El cálculo de distancias entre puntos en el espacio 3D es una operación fundamental. La distancia euclidiana entre dos puntos se calcula usando el teorema de Pitágoras extendido a tres dimensiones:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$$

En Godot, puedes calcular la distancia entre dos puntos usando `point_a.distance_to(point_b)`. Para optimización, si solo necesitas comparar distancias (como encontrar el objeto más cercano), puedes usar `distance_squared_to()`, que evita la costosa operación de raíz cuadrada.

El cálculo de ángulos entre vectores se basa en el producto escalar. El ángulo entre dos vectores se puede calcular usando:

$$\theta = \arccos \left(\frac{\mathbf{a} \cdot \mathbf{b}}{|\mathbf{a}| |\mathbf{b}|} \right)$$

Godot proporciona el método `angle_to()` que calcula directamente el ángulo entre dos vectores. Este cálculo es útil para determinar campos de visión, ángulos de rotación necesarios, y para muchos algoritmos de inteligencia artificial.

Es importante entender que `angle_to()` siempre devuelve un ángulo entre 0 y π radianes (0 a 180 grados). Si necesitas el ángulo con signo, puedes usar el producto vectorial para determinar la dirección de la rotación.

2.2. Transformaciones 3D

Las transformaciones 3D son operaciones que modifican la posición, rotación, y escala de objetos en el espacio tridimensional. Entender estas transformaciones es crucial para crear cualquier aplicación 3D, desde juegos simples hasta simulaciones complejas.

Una transformación se puede representar matemáticamente como una matriz 4x4 que, cuando se multiplica por un vector de posición, produce la nueva posición transformada. Godot encapsula estas operaciones en el tipo `Transform3D`, que hace que trabajar con transformaciones sea más intuitivo.

El orden de las transformaciones es importante. Rotar un objeto y luego trasladarlo produce un resultado diferente que trasladarlo y luego rotarlo. Godot aplica las transformaciones en el orden: escala, rotación, traslación.

2.2.1. Espacios Global y Local

Uno de los conceptos más importantes en gráficos 3D es la diferencia entre espacios de coordenadas globales y locales. El espacio global es el sistema de coordenadas del mundo, mientras que el espacio local es relativo a un objeto específico.

Cuando trabajas con nodos en Godot, cada nodo tiene su propio espacio local. Las transformaciones aplicadas a un nodo se expresan en términos de su espacio local, pero el motor las convierte automáticamente al espacio global para el renderizado.

Esta distinción es fundamental para entender cómo funciona la jerarquía de nodos. Un nodo hijo hereda las transformaciones de su padre, pero sus propias transformaciones se aplican en su espacio local. Esto permite crear sistemas complejos donde, por ejemplo, los dedos de una mano se mueven relativamente a la mano, que se mueve relativamente al brazo.

Godot proporciona métodos para convertir entre espacios: `to_global()` convierte una posición local a global, mientras que `to_local()` hace la conversión inversa. Estos métodos son esenciales cuando necesitas calcular posiciones relativas entre objetos en diferentes partes de la jerarquía.

2.2.2. Rotaciones y Cuaterniones

Las rotaciones en 3D son notoriamente complejas debido a problemas como el gimbal lock y la interpolación no lineal. Godot maneja estas complejidades usando cuaterniones, que son una representación matemática de rotaciones que evita estos problemas.

Un quaternion es una extensión de los números complejos que consta de cuatro componentes: x , y , z , y w . Aunque la matemática detrás de los quaterniones es compleja, Godot proporciona una interfaz intuitiva para trabajar con ellos.

Los quaterniones tienen varias ventajas sobre otras representaciones de rotación:

- No sufren gimbal lock
- Permiten interpolación suave entre rotaciones
- Son más eficientes computacionalmente para composición de rotaciones
- Requieren menos memoria (4 componentes vs 9 para matrices)

En Godot, raramente trabajas directamente con quaterniones. En su lugar, usas métodos como `rotate()`, `rotate_x()`, `rotate_y()`, y `rotate_z()` que internamente usan quaterniones. Para rotaciones más complejas, puedes usar `look_at()` para orientar un objeto hacia un punto específico.

2.2.3. Interpolación Lineal (lerp/slerp)

La interpolación es el proceso de encontrar valores intermedios entre dos puntos conocidos. En animación y movimiento, la interpolación te permite crear transiciones suaves entre estados.

La interpolación lineal (lerp) es la forma más simple de interpolación. Para interpolar entre dos valores a y b con un factor t entre 0 y 1:

$$\text{lerp}(a, b, t) = a + t(b - a) = (1 - t)a + tb$$

Godot proporciona métodos `lerp()` para vectores y otros tipos. Esta función es fundamental para crear animaciones suaves, como mover un objeto gradualmente de una posición a otra.

Para rotaciones, la interpolación lineal no funciona bien porque las rotaciones no forman un espacio lineal. En su lugar, usamos interpolación esférica (slerp), que interpola a lo largo de la superficie de una esfera.

$$\text{slerp}(q_1, q_2, t) = \frac{\sin((1-t)\theta)}{\sin(\theta)} q_1 + \frac{\sin(t\theta)}{\sin(\theta)} q_2$$

donde θ es el ángulo entre los cuaterniones.

En Godot, puedes usar `slerp()` para interpolar suavemente entre rotaciones. Esto es especialmente útil para crear animaciones de rotación naturales, como girar gradualmente un objeto para que mire hacia un objetivo.

2.3. Geometría Computacional

La geometría computacional es el campo de las matemáticas que se ocupa de algoritmos para resolver problemas geométricos. En el desarrollo de juegos, estos algoritmos son fundamentales para tareas como detección de colisiones, generación de terrenos, y renderizado eficiente.

Los algoritmos de geometría computacional a menudo involucran conceptos como convexidad, triangulación, y partición espacial. Aunque Godot implementa muchos de estos algoritmos internamente, entender los principios básicos te ayudará a usar estas herramientas más efectivamente.

2.3.1. Triangulación de Delaunay

La triangulación de Delaunay es un método para dividir un conjunto de puntos en triángulos de manera que ningún punto esté dentro del círculo circunscrito de ningún triángulo. Esta propiedad hace que la triangulación sea "óptima" en varios sentidos.

La triangulación de Delaunay maximiza el ángulo mínimo de todos los triángulos, lo que produce triángulos más regulares y evita triángulos muy delgados. Esto es importante para aplicaciones como generación de mallas y elementos finitos.

En el desarrollo de juegos, la triangulación de Delaunay es útil para:

- Generar terrenos naturales a partir de puntos de altura
- Crear navegación de IA usando grafos de triángulos
- Generar mallas procedurales
- Optimizar la renderización de superficies complejas

El algoritmo más común para calcular la triangulación de Delaunay es el algoritmo de Bowyer-Watson, que añade puntos uno por uno y mantiene la propiedad de Delaunay actualizando los triángulos afectados.

2.3.2. Diagrama de Voronoi

El diagrama de Voronoi es la dualidad geométrica de la triangulación de Delaunay. Para un conjunto de puntos semilla, el diagrama de Voronoi divide el plano en regiones, donde cada región contiene todos los puntos más cercanos a una semilla específica.

Cada celda de Voronoi contiene exactamente una semilla y consiste en todos los puntos que están más cerca de esa semilla que de cualquier otra. Los bordes de las celdas son los lugares donde dos o más semillas están equidistantes.

En el desarrollo de juegos, los diagramas de Voronoi tienen múltiples aplicaciones:

- Generación procedural de mapas y territorios
- Simulación de crecimiento orgánico (como células o colonias)
- Distribución espacial de recursos o enemigos
- Creación de patrones naturales en texturas

La relación entre diagramas de Voronoi y triangulación de Delaunay es fundamental: cada arista de Delaunay corresponde a un borde de Voronoi, y cada triángulo de Delaunay corresponde a un vértice de Voronoi.

2.3.3. Recorte de Polígonos

El recorte de polígonos es el proceso de encontrar la intersección entre dos polígonos o entre un polígono y una región de recorte. Este algoritmo es fundamental para muchas operaciones gráficas.

El algoritmo de Sutherland-Hodgman es uno de los métodos más conocidos para recortar polígonos. Funciona recortando el polígono contra cada borde de la región de recorte secuencialmente.

En el desarrollo de juegos, el recorte de polígonos es útil para:

- Culling de frustum (eliminar objetos fuera del campo de visión)
- Detección de colisiones entre formas complejas
- Generación de sombras y oclusión
- Operaciones booleanas en geometría

El recorte también es importante para optimización. Al eliminar partes de objetos que no son visibles, puedes reducir significativamente la carga de renderizado.

2.3.4. Punto en Polígono

Determinar si un punto está dentro de un polígono es un problema fundamental en geometría computacional. Existen varios algoritmos para resolver este problema, cada uno con diferentes ventajas según el tipo de polígono.

El algoritmo de ray casting es uno de los más intuitivos. Funciona disparando un rayo desde el punto en cualquier dirección y contando cuántas veces cruza los bordes del polígono. Si el número es impar, el punto está dentro; si es par, está fuera.

$$\text{inside} = \sum_{i=0}^{n-1} \text{crossings}(\text{ray}, \text{edge}_i) \bmod 2$$

Para polígonos convexos, puedes usar métodos más eficientes basados en productos vectoriales. Un punto está dentro de un polígono convexo si está en el lado correcto de todos los bordes.

En el desarrollo de juegos, el test de punto en polígono es útil para:

- Detección de clics en interfaces de usuario complejas
- Determinación de zonas de juego y límites
- Sistemas de navegación y pathfinding
- Detección de colisiones 2D

El rendimiento del algoritmo depende del número de vértices del polígono y de si el polígono es convexo o cóncavo. Para polígonos complejos con muchos vértices, puede ser beneficioso usar estructuras de datos espaciales para acelerar las consultas.

Parte III

Sistema de Comportamiento del Gato

Capítulo 3

Sistema de Locomoción Procedural

Ahora que conocemos los conceptos técnicos de Godot y la matemática necesaria para entender el proyecto, podemos empezar a comprender la naturaleza de este sistema y cómo se logró simular a un gato desplazándose por un terreno procedural. No se trata de un sistema estático que sigue una animación predefinida, sino de un sistema dinámico que calcula en cada instante cómo debe moverse el cuerpo del gato según el entorno.

3.1. Control de Movimiento mediante Nodo Objetivo

Primero hay que preguntarse: ¿cómo logramos que el gato se mueva hacia un lugar sin tener que controlar cada paso? Calcular todos los movimientos a mano sería muy costoso y poco flexible, sobre todo en un mundo que se genera de forma dinámica. Por eso, se decidió crear un nodo central llamado **objetivo**, que define la posición deseada hacia la que debe caminar el gato. Este nodo puede colocarse, por ejemplo, haciendo clic en el terreno. A partir de ahí, en cada fotograma se recalcula la posición que debería tener cada pata, tomando en cuenta la geometría del entorno.

3.2. Cinemática Inversa (IK) para las Patas del gato

Ahora, con esas posiciones ya definidas, surge otro problema: mover cada pata sin tener que manipular directamente todos los huesos involucrados. Aquí entra en juego la cinemática inversa (IK, por sus siglas en inglés). Esta técnica permite que las articulaciones del esqueleto se ajusten automáticamente para que una parte —en este caso, la pata— llegue a un punto específico en el espacio. Es como decirle al sistema “coloca la pata aquí”, y él calcula cómo deben moverse el fémur, la tibia y los demás huesos para lograrlo. Godot facilita este proceso con nodos IK que se encargan de esa solución por nosotros.

3.3. Orientación Automática del Cuerpo

Pero, ¿qué pasa si el objetivo está delante, pero el cuerpo del gato sigue mirando hacia otro lado? El movimiento se vería forzado y poco natural. Para evitar eso, el cuerpo también se ajusta automáticamente para alinearse con la dirección del movimiento. Se calcula el ángulo entre su orientación actual y el vector que apunta hacia el objetivo, y se rota suavemente el modelo para que siempre esté orientado correctamente.



Figura 3.1: Modelo del gato moviéndose hacia el objetivo (nodo rojo), el siguiente movimiento de sus patas está dado por los nodos azules.

3.4. Simulación de la Columna Vertebral con Resorte Amortiguado

Hasta aquí tenemos las patas funcionando bien, pero todavía falta algo esencial: conectar todo eso con una columna vertebral que mantenga la estructura y el equilibrio del cuerpo. Mover cada hueso de la columna manualmente tampoco es una buena idea. En su lugar, se implementó una solución inspirada en física: se modeló la cabeza del gato como si estuviera unida al cuerpo por un resorte amortiguado. Esto permite que la cabeza se mueva con un pequeño retraso, oscilando ligeramente con cada paso, como ocurre en un animal real.

La fuerza del resorte que regula este movimiento se basa en la siguiente fórmula:

$$F = -k \cdot x - b \cdot v$$

donde x es el desplazamiento desde la posición objetivo, v es la velocidad, k es la rigidez del resorte y b es el coeficiente de amortiguamiento. Esta fuerza se traduce en una aceleración que actualiza la posición de la cabeza a lo largo del tiempo.

Listing 3.1: Simulación de resorte para la cabeza

```
var displacement = current_pos - desired_pos
var spring_force = -spring_stiffness * displacement - spring_damping *
    velocity
var acceleration = spring_force / mass
velocity += acceleration * delta
current_head_pos += velocity * delta
```

Esta técnica se conoce como integración de Verlet, y permite simular el efecto de inercia de forma fluida y realista sin necesidad de activar el motor de física completo. Así, el movimiento de la cabeza se ve natural, con un pequeño “rebote” que transmite peso y respuesta muscular.

3.5. Adaptación de la Columna Vertebral

Con la cabeza actualizada, el resto de la columna se adapta hacia atrás, resolviendo cada hueso desde la cabeza hasta el torso. Para mantener la coherencia del esqueleto, se calcula la dirección entre cada par de huesos y se proyecta sobre su plano local, aplicando rotaciones suaves que mantienen la postura sin torsiones bruscas.

Listing 3.2: Orientación progresiva de los huesos

```
var dir = positions[i+1] - positions[i]
var dir_proj_zx = Vector3(dir.x, 0, dir.z)
pos.basis = base_basis * Basis(Vector3.BACK,
    Vector3(0,0,10).signed_angle_to(dir_proj_zx, Vector3.DOWN))
```

3.6. Animación de la Cola

La cola del gato también tiene su propia lógica. A diferencia de la columna, no usa un resorte. En su lugar, emplea una interpolación lineal (lerp) que hace que la cola se acerque progresivamente a su posición deseada. Esto genera un movimiento fluido y retardado que imita la forma en que una cola real responde al movimiento del cuerpo: no de golpe, sino con suavidad y una ligera oscilación que aporta expresividad.

3.7. Coordinación entre patas: Magnet Points

Por último, para que las patas traseras se comporten de forma más convincente, se usan puntos de atracción llamados “magnet points”. Estos puntos están ubicados en las patas delanteras, y las traseras se alinean hacia ellos mediante una cadena de IK extendida. De esta forma, se genera una coordinación sutil entre las patas, como si el gato supiera de forma instintiva hacia dónde debería llevar la zanca. Esta técnica mejora el ritmo del paso y refuerza la sensación de unidad en el movimiento.

En conjunto, todos estos elementos logran que el cuerpo del gato se adapte constantemente al entorno sin perder coherencia física ni estética. Es un sistema

modular, flexible y lo suficientemente robusto como para funcionar en un mundo que cambia en tiempo real.

3.8. Patrón de Caminata Cuadrúpeda

¿Cómo hacer que un gato virtual camine de forma natural sin que parezca un robot torpe? Los animales cuadrúpedos no mueven las cuatro patas al mismo tiempo, y si intentáramos programar eso, el gato se vería completamente antinatural. El desafío está en reproducir la secuencia específica que usan los felinos reales para mantener el equilibrio mientras avanzan. La clave está en observar que los gatos (y la mayoría de cuadrúpedos) siguen un patrón diagonal: mueven una pata delantera, luego la pata trasera del lado opuesto, después la otra pata delantera, y finalmente la pata trasera restante. Este patrón asegura que siempre haya al menos tres patas tocando el suelo, manteniendo la estabilidad. Para implementar esto, se creó una máquina de estados con cinco fases de movimiento: PASO_1, PASO_2, PASO_3, PASO_4 y PASO_5. Cada estado define qué pata debe moverse en ese momento y cuál es su objetivo espacial. La transición entre estados ocurre cuando la pata activa alcanza su destino, activando automáticamente el movimiento de la siguiente pata en la secuencia. Esto garantiza un flujo continuo y natural del movimiento, donde el gato nunca pierde el equilibrio y siempre avanza de manera coordinada.

3.9. Secuencia de Movimiento de Patas

Ahora surge un problema de coordinación: ¿cómo sincronizar cuatro extremidades independientes para que trabajen juntas? Si cada pata se moviera por su cuenta sin coordinación, el gato podría terminar en posiciones imposibles o perder el equilibrio. Necesitamos un sistema que permita control individual de cada pata pero manteniendo la coherencia del conjunto. La solución radica en tratar cada pata como un actuador independiente con su propio objetivo, pero bajo el control de un coordinador central que decide cuándo cada una debe activarse. Es como un director de orquesta que permite a cada músico tocar su parte, pero coordinando cuándo debe empezar cada uno. El sistema utiliza un diccionario llamado `objetivos` que especifica las coordenadas exactas donde debe posicionarse cada pata, y otro diccionario `progreso_movimiento` que rastrea qué tan cerca está cada extremidad de alcanzar su meta. Cuando una pata completa su trayectoria (*progreso* $\geq 0,9$), el sistema evalúa automáticamente si es momento de activar la siguiente pata según el patrón de caminata. Cada pata se mueve de forma independiente hacia su objetivo, pero la secuencia general está perfectamente coordinada por la máquina de estados.

3.10. Interpolación Parabólica de Patas

¿Por qué las patas del gato no pueden simplemente deslizarse en línea recta de un punto a otro? En la realidad, las patas se levantan del suelo, forman un arco en el aire, y luego se depositan en el nuevo punto. Si usáramos interpolación lineal, las

patas parecerían arrastrarse por el suelo, lo cual se ve muy poco natural. La idea es recrear este movimiento arqueado usando una función parabólica que simule el levantamiento y descenso natural de la pata. El movimiento debe partir del punto inicial, alcanzar una altura máxima en el punto medio, y descender suavemente al destino final.

Listing 3.3: Cálculo de altura parabólica

```
func calcular_altura_parabola(pos_inicial, pos_final, progreso,
    factor_altura = 1.0):
var altura_maxima = max(y_inicio, y_fin) + (ALTURA_PASO * factor_altura)
var a = y_inicio + y_fin - 2 * altura_maxima
var b = -2 * y_inicio + 2 * altura_maxima
var c = y_inicio
return a * progreso * progreso + b * progreso + c
```

La función calcula dinámicamente los coeficientes de una parábola que pasa por el punto inicial y final, pero alcanza una altura máxima predefinida en el centro. El `factor_altura` permite ajustar qué tan pronunciado es el arco dependiendo del tipo de movimiento: pasos normales usan valores bajos, mientras que saltos requieren arcos mucho más altos para superar obstáculos.

3.11. Sistema de Cohesión de Patas

Un problema grave surge cuando las patas intentan posicionarse en lugares anatómicamente imposibles: ¿qué pasa si la pata izquierda trata de cruzarse con la derecha, o si una pata intenta alejarse tanto que el gato se “estire” de forma irreal? Sin restricciones, el gato podría acabar en configuraciones que romperían completamente la ilusión de realismo. La estrategia es establecer límites espaciales que respeten la anatomía felina real. Cada pata debe mantenerse dentro de un rango razonable respecto al centro del cuerpo, y las patas del mismo lado no pueden cruzarse entre sí. Es como poner reglas de tráfico para las extremidades. El sistema implementa la función `validar_cohesion_patas` que verifica continuamente la distancia de cada pata respecto al centro del cuerpo. Si una pata intenta posicionarse más allá de `DISTANCIA_MAXIMA_PATA`, el algoritmo calcula la posición válida más cercana proyectando el punto hacia el límite permitido. También verifica que las patas izquierda y derecha mantengan una separación mínima para evitar cruzamientos. Cuando se detecta una violación, la posición se corrige automáticamente, y se recalcula la altura del terreno en la nueva posición para asegurar que la pata termine en contacto correcto con el suelo.

3.12. Sistema de Salto Adaptativo

¿Qué hacer cuando el gato se encuentra con un obstáculo que no puede superar caminando normalmente? Un escalón alto, un hueco en el tejado, o una elevación pronunciada requieren un comportamiento completamente diferente al caminar. El

gato necesita poder saltar, pero no de forma fija, sino adaptándose al tipo y tamaño del obstáculo. La clave está en reconocer que existen dos tipos fundamentales de obstáculos: elevaciones (saltar hacia arriba) y depresiones (saltar hacia abajo). Cada uno requiere una estrategia diferente de preparación, ejecución y aterrizaje. Un salto hacia arriba necesita más impulso vertical, mientras que uno hacia abajo requiere más control en el aterrizaje. El sistema de salto se estructura en tres fases distintas manejadas por estados específicos: `SALTO_PREP`, `SALTO` y `ATERRIZAJE`. Durante la preparación, todas las patas se agrupan en una posición compacta cerca del centro del cuerpo, creando una plataforma estable para generar impulso. En la fase de ejecución, las patas delanteras se extienden primero hacia el punto de aterrizaje, seguidas por las traseras. Los factores de altura parabólica se ajustan dinámicamente: `factor_altura = 0.5` para saltos hacia abajo (movimiento más controlado) y `factor_altura = 3.0` para saltos hacia arriba (arco más pronunciado para superar el obstáculo).

3.13. Detección de Obstáculos

¿Cómo puede el gato "ver" los obstáculos antes de chocar con ellos? Sin un sistema de detección anticipada, el gato seguiría caminando hasta encontrarse físicamente con una pared o hueco, lo que se vería muy poco inteligente. Necesitamos que el gato "mire hacia adelante" identifique problemas en su trayectoria. La idea es implementar una "visión hacia adelante" que proyecte rayos desde la posición actual del gato hacia la dirección de movimiento, verificando si hay cambios significativos en la altura del terreno que indiquen la presencia de obstáculos.

Listing 3.4: Detección de cambios de altura críticos

```
for dist in range(1, 30):
var distancia_actual = UMBRAL_DETECCION_SALTO * dist / 30.0
var punto_check = centro + dir_norm * distancia_actual
var altura_terreno = obtener_punto_mas_alto(punto_check.x, punto_check.z,
"suelo").y
var diferencia_altura = altura_terreno - centro.y

if diferencia_altura > MIN_HEIGHT_JUMP && diferencia_altura <
MAX_HEIGHT_JUMP:
    posicion_obstaculo = Vector3(punto_check.x, altura_terreno,
    punto_check.z)
    return true
```

El algoritmo escanea incrementalmente el terreno adelante del gato, dividiendo la distancia de detección en 30 segmentos. En cada punto, calcula la altura del suelo y la compara con la altura actual del gato. Si encuentra una diferencia que está dentro del rango de capacidad de salto (`MIN_HEIGHT_JUMP` a `MAX_HEIGHT_JUMP`), marca esa posición como obstáculo y activa el sistema de salto. Si la diferencia es demasiado grande, el gato busca una ruta alternativa.

3.14. Cálculo de Trayectoria de Salto

Una vez detectado un obstáculo, surge el problema de calcular exactamente dónde debe saltar el gato: ¿dónde está el borde exacto del obstáculo? ¿cuál es la trayectoria óptima para superarlo? Un cálculo impreciso podría hacer que el gato salte demasiado pronto o demasiado tarde. La estrategia es localizar con precisión el borde del obstáculo usando búsqueda binaria, refinando iterativamente entre el último punto seguro y el primer punto problemático. Una vez encontrado el borde exacto, se calcula una trayectoria que permita aterrizar de forma segura más allá del obstáculo. El algoritmo comienza con dos puntos conocidos: `ultimo_punto_bajo` (donde el gato aún puede caminar) y `primer_punto_alto` (donde comienza el obstáculo). Mediante búsqueda binaria, calcula puntos medios sucesivos hasta localizar el borde con precisión. Para saltos hacia arriba, busca el punto donde la altura supera `MIN_HEIGHT_JUMP`; para saltos hacia abajo, donde cae por debajo de ese umbral. Una vez identificado el borde, calcula la posición de aterrizaje añadiendo la longitud natural del paso del gato más un margen de seguridad (`MARGEN_SALTO_ADELANTE`). La altura de la trayectoria se ajusta para garantizar que pase cómodamente por encima del obstáculo.

3.15. Preparación y Ejecución de Salto

¿Cómo lograr que el salto se vea natural y poderoso, como el de un gato real? Los animales no saltan desde cualquier posición; primero recolectan sus patas, agrupándose para crear una plataforma estable, y luego se extienden coordinadamente para generar impulso direccional. La idea es replicar esta preparación natural: primero todas las patas se mueven hacia una configuración compacta que permita generar máximo impulso, luego se ejecuta la extensión en dos fases coordinadas (delanteras primero, traseras después). Durante `SALTO_PREP`, el sistema calcula una posición base ligeramente adelantada del centro actual y agrupa todas las patas en una configuración compacta alrededor de este punto. Las patas delanteras se posicionan con una separación del 40 % de la distancia normal, y las traseras al 20 %, creando una postura "gazapada" lista para saltar. En `SALTO`, solo las patas delanteras se mueven hacia el punto de aterrizaje calculado, mientras las traseras mantienen su posición para generar impulso. Finalmente, en `ATERRIZAJE`, las patas traseras siguen a las delanteras, completando el movimiento. Las velocidades se ajustan según el tipo de salto: `VELOCIDAD_PREPARACION_SALTO` = 0.5 para movimientos precisos durante la preparación, y `VELOCIDAD_EJECUCION_SALTO` = 0.8 para la fase dinámica del salto.

3.16. Detección y Adaptación al Terreno

¿Cómo hacer que las patas del gato siempre toquen el suelo correctamente, sin importar lo irregular que sea el terreno? En un mundo procedural con tejados, escalones y superficies inclinadas, cada pata debe encontrar independientemente su punto de contacto óptimo con la superficie. La aproximación es usar el motor de física del juego para "preguntar" al terreno cuál es su altura exacta en cualquier

punto dado. Cada vez que el sistema calcula dónde debe ir una pata, también debe verificar a qué altura está el suelo en esa posición específica. El sistema utiliza la función `obtener_punto_mas_alto` que dispara un rayo vertical desde bien arriba del terreno (30 unidades por encima del gato) hacia abajo hasta muy por debajo (-1000 unidades). Este rayo intersecta con cualquier objeto marcado como "suelo" y devuelve las coordenadas exactas del punto de contacto. El proceso se ejecuta continuamente para cada objetivo de pata, asegurando que las posiciones siempre correspondan a puntos válidos en la superficie. Cuando una pata se mueve hacia su objetivo, su coordenada Y se actualiza constantemente con la altura real del terreno en esa posición, garantizando un contacto perfecto independientemente de la irregularidad de la superficie.

3.17. Raycast para Altura de Suelo

¿Cómo determinar con precisión matemática la altura exacta del suelo en cualquier punto del terreno irregular? Este es un problema fundamental porque todo el sistema de movimiento depende de conocer exactamente dónde termina el aire y comienza la superficie sólida. La solución utiliza el concepto de raycast: disparar un rayo invisible desde un punto conocido hacia una dirección específica, y detectar dónde intersecta con la geometría del mundo. Es como usar un láser para medir distancias, pero en el mundo virtual. La implementación configura cuidadosamente los parámetros del raycast usando `PhysicsRayQueryParameters3D`. El origen se posiciona 30 unidades por encima de la posición objetivo (para asegurar que esté en el aire), y el destino se coloca a -1000 unidades (muy por debajo de cualquier terreno esperado). El sistema filtra las colisiones para considerar únicamente objetos que pertenezcan al grupo "suelo", evitando interferencias con decoraciones, otros personajes, o elementos no transitables. Cuando el rayo intersecta con un objeto válido, `resultado["position"]` contiene las coordenadas exactas del punto de contacto, incluyendo la altura precisa que necesita el sistema de movimiento.

3.18. Adaptación de Altura de Patas

¿Cómo asegurar que cada pata mantenga contacto apropiado con superficies que cambian constantemente de altura? Si las patas no se adaptan a la topografía del terreno, el gato parecería flotar sobre algunas áreas o hundirse en otras. La idea es recalcular continuamente la altura objetivo de cada pata basándose en lecturas en tiempo real del terreno en su posición específica. Cada pata debe "preguntar" al terreno cuál es su altura en el momento exacto en que va a hacer contacto. El sistema actualiza la coordenada Y de cada objetivo en `objetivos[nombre_pata].y` usando `obtener_punto_mas_alto` en las coordenadas X y Z de destino. Este proceso ocurre no solo cuando se establece un nuevo objetivo, sino también durante la interpolación de movimiento. Mientras la pata se mueve en su trayectoria parabólica, el sistema recalcula constantemente dónde debería terminar su arco basándose en la altura real del terreno en el punto de aterrizaje. Esto significa que si el terreno cambia durante el movimiento (por ejemplo, en un mundo dinámico), la pa-

ta automáticamente ajusta su trayectoria para aterrizar correctamente. La función `calcular_posicion_interpolada` combina el movimiento horizontal suave con la adaptación vertical precisa, asegurando que cada paso termine en contacto perfecto con la superficie.

3.19. Validación de Próximos Pasos

¿Qué pasa si el gato está a punto de dar un paso hacia un precipicio o una zona inaccesible? Sin validación previa, el gato podría intentar moverse hacia posiciones imposibles o peligrosas, quedando atrapado o cayendo al vacío. La estrategia es "simular" mentalmente el próximo paso antes de ejecutarlo, verificando que tanto la posición de aterrizaje como la trayectoria intermedia sean viables según las capacidades físicas del gato. Es como mirar antes de dar el paso.

Listing 3.5: Validación de próximo paso

```
func validar_siguiente_paso() -> bool:
var pos_frontL = centro + dir_norm * LONGITUD_PASO + Vector3(direccion.z,
    0, -direccion.x) * (DISTANCIA_ENTRE_PATAS / 2)
var altura_L = obtener_punto_mas_alto(pos_frontL.x, pos_frontL.z, "suelo")
    .y

if altura_L < centro.y - UMBRAL_ALTURA_CRITICA:
    var pos_borde = buscar_posicion_borde_seguro(centro + dir_norm *
        LONGITUD_PASO)
    return pos_borde != Vector3.ZERO
```

El algoritmo proyecta las posiciones futuras de las patas delanteras basándose en la dirección actual y la longitud de paso estándar. Luego verifica que estas posiciones tengan alturas apropiadas del terreno, comparándolas con `UMBRAL_ALTURA_CRITICA`. Si detecta que el próximo paso sería hacia un precipicio o una superficie demasiado alta/baja, activa `buscar_posicion_borde_seguro` que explora direcciones alternativas en un rango de 60 grados a cada lado. Si encuentra una ruta viable, ajusta la dirección; si no, activa el sistema de salto o busca comportamientos alternativos para superar el obstáculo.

3.20. Estados de Caminata (PASO_1 a PASO_5)

¿Cómo puede un sistema computacional replicar la coordinación natural que un gato real ejecuta instintivamente al caminar? El desafío radica en que la caminata cuadrúpeda requiere una secuencia específica de movimientos que debe mantener el equilibrio mientras produce un avance suave y natural. La idea fundamental consiste en dividir el ciclo completo de caminata en estados discretos, donde cada estado representa una fase específica del patrón de locomoción. Cada estado define exactamente qué pata debe moverse y cuándo, creando una secuencia ordenada que respeta la biomecánica felina natural. El sistema implementa cinco estados principales de caminata: `PASO_1` actúa como estado base donde el gato evalúa si debe continuar

moviéndose. PASO_2 mueve la pata trasera derecha hacia la posición que ocupaba la pata delantera derecha, creando estabilidad posterior. PASO_3 avanza la pata delantera derecha hacia una nueva posición adelantada, extendiendo el alcance del gato. PASO_4 replica el movimiento trasero pero en el lado izquierdo, llevando la pata trasera izquierda hacia donde estaba la delantera izquierda. Finalmente, PASO_5 completa el ciclo moviendo la pata delantera izquierda hacia adelante, preparando el terreno para reiniciar la secuencia. Cada transición entre estados ocurre cuando la pata activa alcanza su objetivo, medido a través del sistema `progreso.movimiento` que rastrea qué tan cerca está cada extremidad de su destino. Cuando el progreso supera el umbral de 0.9 (90 % completado), el sistema automáticamente activa el siguiente estado en la secuencia. Esta aproximación asegura que nunca se interrumpa un movimiento a medias y que siempre haya al menos tres patas proporcionando estabilidad mientras una se encuentra en tránsito.

3.21. Estados de Salto (PREP, SALTO, ATERRI- ZAJE)

¿Qué sucede cuando el patrón de caminata normal no es suficiente para superar un obstáculo? Los gatos no pueden simplemente continuar caminando cuando se enfrentan a escalones altos, huecos o cambios pronunciados de elevación. Necesitan una secuencia de comportamiento completamente diferente que les permita superar estas barreras de forma segura y eficiente. La estrategia radica en reconocer que el salto es fundamentalmente diferente a la caminata: requiere una fase de preparación donde el gato "se agacha" para generar impulso, una fase de ejecución donde extiende su cuerpo para alcanzar el destino, y una fase de aterrizaje donde recupera el control y la estabilidad en la nueva posición. SALTO_PREP representa la fase de preparación donde todas las patas se agrupan en una configuración compacta cerca del centro del cuerpo. Este estado calcula una posición base ligeramente adelantada y posiciona las patas delanteras con el 40 % de separación normal y las traseras con el 20 %, creando una postura "agazapada" que simula cómo los felinos reales se preparan antes de saltar. La velocidad se reduce a `VELOCIDAD_PREPARACION.SALTO = 0.5` para permitir posicionamiento preciso y dar tiempo al sistema para calcular la trayectoria óptima. SALTO constituye la fase de ejecución dinámica donde solo las patas delanteras se extienden hacia el punto de aterrizaje calculado, mientras las traseras mantienen su posición para simular el impulso generado desde la posición agazapada. La velocidad aumenta a `VELOCIDAD_EJECUCION.SALTO = 0.8` para crear la sensación de movimiento explosivo característico del salto felino. Durante esta fase, el sistema utiliza factores de altura parabólica ajustados dinámicamente según el tipo de obstáculo detectado. ATERRIZAJE completa la secuencia llevando las patas traseras a las mismas posiciones que las delanteras, simulando cómo el gato recoge sus extremidades posteriores para completar el salto. La velocidad se ajusta a `VELOCIDAD_ATERRIZAJE = 0.8` para mantener fluidez pero permitir control preciso del contacto con el suelo. Una vez que todas las patas alcanzan sus objetivos, el sistema transiciona automáticamente de vuelta al estado PASO_1 para continuar la locomoción normal.

3.22. Estados de Carrera

¿Cómo debe comportarse el gato cuando el objetivo está muy lejos y la caminata normal resultaría demasiado lenta? Los felinos reales cambian su patrón de locomoción cuando necesitan cubrir distancias grandes rápidamente, adoptando un estilo de "galope" que prioriza la velocidad sobre la precisión del movimiento. La conceptualización de la carrera reconoce que este comportamiento requiere saltos más largos y frecuentes, con menos tiempo dedicado a la precisión del posicionamiento de patas. El gato debe poder mantener este patrón de alta velocidad mientras sigue siendo capaz de detectar y reaccionar a obstáculos. `CORRER.PREP` funciona de manera similar a `SALTO.PREP` pero con parámetros optimizados para velocidad. Las patas se agrupan en una configuración ligeramente menos compacta y el tiempo de preparación se reduce significativamente mediante una velocidad incrementada de $VELOCIDAD_PREPARACION_SALTO * VELOCIDAD_BASE * VELOCIDAD_CORRER * 3$. El factor multiplicador de 3 refleja la urgencia del movimiento de carrera. `CORRER.SALTO` ejecuta saltos de distancia variable calculada dinámicamente por `calcular_longitud_salto_correr()`, que interpola entre `SALTO_CORRER_MIN = 180 * ESCALA` y `SALTO_CORRER_MAX = 400 * ESCALA` basándose en la distancia al objetivo. Cuando el gato está más lejos del objetivo, los saltos son más largos para cubrir distancia rápidamente; cuando está más cerca, los saltos se acortan para permitir mayor precisión en el acercamiento final. `CORRER.ATERRIZAJE` maneja la recuperación rápida de la posición de salto, pero incluye lógica adicional para determinar si continuar corriendo o transicionar a caminata. Si la distancia al objetivo sigue siendo mayor que `DISTANCIA_PARAR_CORRER = 600 * ESCALA`, el sistema inmediatamente reinicia en `CORRER.PREP` para mantener el patrón de galope. Si la distancia es menor, transiciona a caminata normal para el acercamiento final controlado.

3.23. Lógica de Transiciones

¿Cómo determina el sistema cuándo cambiar de un comportamiento a otro sin crear transiciones abruptas o decisiones contradictorias? La coordinación entre diferentes tipos de locomoción requiere un sistema de decisiones jerárquico que evalúe continuamente las condiciones y seleccione el comportamiento más apropiado. La idea central es implementar un patrón de diseño llamado "máquina de estados jerárquica" donde diferentes comportamientos (caminata, salto, carrera) actúan como estados de alto nivel, cada uno con sus propios sub-estados internos. Un controlador central evalúa las condiciones y maneja las transiciones entre comportamientos principales.

Listing 3.6: Implementación del sistema de transiciones

```
class BehaviorHandler:
func update():
if current_behavior:
var next_behavior = current_behavior.update()
if next_behavior != current_behavior_name:
transition_to(next_behavior)
```

```
func transition\_to(behavior\_name: String): if behavior\_name in
    behaviors and current\_behavior.can\_transition\_to(behavior\_name):
    current\_behavior.exit() start\_behavior(behavior\_name)
```

El sistema `BehaviorHandler` actúa como coordinador central que mantiene referencias a todos los comportamientos disponibles: `WalkBehavior`, `JumpBehavior` y `RunBehavior`. Cada comportamiento implementa métodos estándar: `enter()` para inicialización, `update()` para lógica de frame-a-frame, `exit()` para limpieza, y `can_transition_to()` para validar transiciones permitidas. La lógica de transición en `WalkBehavior` evalúa múltiples condiciones en orden de prioridad: primero verifica si la distancia al objetivo excede `DISTANCIA_PARA_CORRER = 1000 * ESCALA`, activando carrera para distancias largas. Si no está corriendo, verifica si `es_necesario_saltar()` detecta obstáculos que requieren salto. Solo si ninguna de estas condiciones especiales se cumple, continúa con la lógica normal de caminata. `JumpBehavior` maneja transiciones internas entre sus sub-estados de salto, pero solo puede transicionar de vuelta a `WalkBehavior` una vez completada toda la secuencia de salto. Esto previene interrupciones peligrosas a mitad de un salto que podrían dejar al gato en una posición inestable. `RunBehavior` evalúa continuamente si la distancia al objetivo ha caído por debajo de `DISTANCIA_PARAR_CORRER`, transicionando automáticamente a caminata cuando el gato se acerca lo suficiente al destino. También incluye lógica para manejar obstáculos durante la carrera, ajustando dinámicamente la altura de salto mediante `salto_es_alto` basándose en la diferencia de elevación detectada. El sistema de validación `can_transition_to()` implementa reglas de seguridad que previenen transiciones problemáticas: los saltos no pueden interrumpirse una vez iniciados, la carrera no puede activarse si hay obstáculos inmediatos que requieren precisión, y la caminata normal no puede forzarse si el gato está en medio de un movimiento de alta velocidad que requiere completarse antes de cambiar de comportamiento.

Capítulo 4

Sistema de Animación de Cola

La cola del gato es un elemento fundamental para su comportamiento. En los gatos, la cola actúa como un indicador emocional, por lo que se mueve de forma diferente dependiendo de lo que el gato esté “pensando”. Precisamente para simular ese “pensamiento”, utilizamos máquinas de estados que representan la emoción o estado actual del gato.

En este capítulo implementaremos un sistema completo de animación de cola que simula estos comportamientos de forma realista.

Aviso

Este sistema está diseñado para integrarse de manera similar a la máquina de estados desarrollada en los capítulos anteriores.

4.1. Comportamiento Real

Antes de programar la cola, es necesario entender cómo se mueve la cola de un gato. Tras la observación, notamos varias características importantes:

- La cola del gato nunca se mantiene completamente inmóvil; siempre hay al menos un movimiento sutil.
- El movimiento varía según la actividad: es distinto si el gato camina, salta, está alerta, etc.
- Los gatos tienen “preferencias” al mover la cola más hacia la izquierda, la derecha, etc.
- La intensidad del movimiento depende del gato, pero casi siempre la cola se mueve como un péndulo (de izquierda a derecha repetidamente).

Podemos entonces modelar la cola como un sistema que reacciona a tres factores principales:

1. Estado emocional interno (¿cómo se siente el gato?).

2. Actividad física (¿qué está haciendo el gato?).
3. Personalidad individual (¿cuál es la personalidad de nuestro gato? ¿Qué tan intensa es?).

Esto nos lleva a un diseño de tres capas:

Estado base → Modificador emocional → Modificador personal → Posición final

4.2. Representación Matemática

Para el movimiento físico, necesitamos representar ondulaciones suaves. Las funciones trigonométricas, como el seno y el coseno, son ideales para este objetivo, ya que producen movimientos cíclicos tipo bamboleo y no son costosas en términos de cómputo.

Decisión de diseño: Usaremos múltiples ondas senoidales combinadas, donde cada una representa un aspecto distinto del comportamiento.

Sin embargo, limitaremos la representación de la cola a un único punto, ya que animar la cola completa sería un problema mucho más complejo. Al enfocarnos solamente en la punta de la cola, el problema se reduce a una simple evaluación de funciones como seno o coseno. Más adelante veremos cómo extender esta lógica al resto de la 'curva' de la cola.

4.3. Sistema de Personalidad

Si todos los gatos se comportaran de la misma manera, no habría personalidad. Dado que la cola representa el estado de ánimo del gato, es necesario reflejar 'cómo es' nuestro gato. Para ello, podemos definir lo que llamamos *offsets de personalidad*:

```
# Generamos una personalidad única una sola vez
cola_direccion_preferida = randf_range(-1.0, 1.0)
cola_offset_personal = Vector3(randf_range(-0.3, 0.3), ...)
```

Estos valores aleatorios son útiles para generar una personalidad única por cada gato. Los rangos se han escogido cuidadosamente para evitar comportamientos extremos o poco realistas.

4.3.1. La personalidad como multiplicador

La personalidad no es solo un *offset*; habíamos mencionado que la cola se comporta como una suma de ondas sinusoidales, y la personalidad influye también en su intensidad. En general, nos indica qué tanto mueve la cola un gato en particular.

Podemos definir las reacciones emocionales de la siguiente manera:

```
lateral += sin(cola_tiempo * 3.2) * 0.2 * FACTOR_PERSONALIDAD * AMPLITUD_LATERAL
```

En este contexto, `lateral` representa el desplazamiento lateral de la cola, es decir, dónde debería estar la punta de la cola en un instante determinado. La variable `FACTOR_PERSONALIDAD` nos indica qué tan “activo” o “vivaz” es nuestro gato. Un valor alto implicaría movimientos más amplios y notorios.

4.3.2. Estados emocionales

La observación del comportamiento felino nos permite identificar distintos estados emocionales, cada uno con características únicas en el movimiento de la cola:

- **Relajada:** Movimiento lento, amplitud baja.
- **Curiosa:** Movimiento medio, con variaciones frecuentes.
- **Alerta:** Movimiento rápido, cola erguida.
- **Concentrada:** Movimiento controlado, con micro-temblores.

Cada estado emocional se traducirá en un conjunto específico de parámetros para nuestras funciones trigonométricas. Estos parámetros son totalmente personalizables. Por ejemplo, para el estado **Relajada** podríamos usar:

- Frecuencias bajas: 0.5, 0.6, 0.8 → movimiento lento.
- Amplitudes reducidas: 0.3, 0.4 → movimiento sutil.
- Ondas simples → suavidad visual.

4.4. Movimiento de la cola

4.4.1. Ondulación lateral

Para cada estado combinaremos múltiples ondas senoidales con diferentes propósitos:

```
# Onda principal (movimiento base)
lateral = sin(cola_tiempo * frecuencia_base) * amplitud_base

# Onda de personalidad (preferencias individuales)
lateral += cola_direccion_preferida * factor_preferencia

# Onda de "capricho" (variabilidad)
lateral += sin(cola_tiempo * frecuencia_alta) * amplitud_capricho
```

¿Por qué tres ondas? Las ondas nos permiten calcular una posición que se siente orgánica y no mecánica. Si solo tuviéramos una onda base, el movimiento sería repetitivo. Al sumar la capa de personalidad, el gato se vuelve único. Finalmente, con la onda de “capricho” se evita que el movimiento sea predecible, introduciendo variabilidad natural.

Las ondulaciones verticales y la profundidad (el eje hacia atrás) pueden calcularse de forma similar.

4.4.2. Movimiento tridimensional

Hasta este punto hemos descrito un sistema que funciona en dos dimensiones para manejar el movimiento lateral de la cola. Sin embargo, en un entorno 3D, la cola no se mueve en un plano fijo, sino en un espacio relativo a la orientación del gato. Esto plantea varios desafíos, como:

- ¿Cómo definimos “lateral” si el gato puede mirar en cualquier dirección?
- ¿Cómo mantenemos la cola “detrás” del gato, independientemente de su rotación?
- ¿Cómo coordinamos la altura, la posición lateral y la profundidad?

La solución es calcular vectores de referencia a partir de la dirección actual del gato. Esto puede hacerse de la siguiente manera:

```
var vector_lateral = Vector3(direccion.z, 0, -direccion.x).normalized()
var vector_atras = -direccion.normalized()
```

Donde `direccion` es el vector que indica hacia dónde está mirando el gato. Para obtener el vector lateral, rotamos la dirección 90 grados en el plano horizontal. El vector hacia atrás simplemente es la dirección invertida.

La posición final de la punta de la cola se calcula sumando las siguientes componentes vectoriales:

$$\vec{P} = \vec{v}_{\text{lateral}} \cdot d_{\text{lateral}} + \begin{bmatrix} 0 \\ h \\ 0 \end{bmatrix} + \vec{v}_{\text{atrás}} \cdot d_{\text{profundidad}}$$

Donde: - $\vec{v}_{\text{lateral}}$: vector perpendicular a la dirección del gato. - d_{lateral} : desplazamiento lateral calculado con ondas. - h : altura calculada por el sistema (movimiento vertical). - $\vec{v}_{\text{atrás}}$: vector hacia atrás. - $d_{\text{profundidad}}$: desplazamiento hacia atrás.

Cada componente se puede calcular de forma independiente y luego combinar vectorialmente para obtener el movimiento tridimensional final de la cola.

4.4.3. Reacción al Movimiento del Cuerpo

La cola no puede existir en el vacío; debe reaccionar a lo que está haciendo el gato y a dónde se está desplazando. Esto requiere comunicación entre el sistema de movimiento y el sistema de animación de la cola. También es necesaria una traducción entre los estados de comportamiento corporal y los estados del sistema de cola.

4.4.4. El patrón *Observer*

Una primera intención podría ser manejar eventos directamente: es decir, que los cambios de estado del movimiento llamen a funciones específicas del sistema de la cola. Sin embargo, esto complicaría notablemente la legibilidad del código.

El patrón *Observer* resuelve esta inconveniencia. En lugar de tener un sistema complejo de eventos, implementamos dos subsistemas: el de movimiento y el de la cola. El sistema de cola simplemente observa al de movimiento todo el tiempo.

```
match behavior_handler.get_current_behavior_name():
    "walk":
        estado_cola_actual = EstadoCola.CURIOSA if debe_avanzar() else
            EstadoCola.RELAJADA
    "run":
        estado_cola_actual = EstadoCola.ALERTA
    "jump":
        estado_cola_actual = EstadoCola.CONCENTRADA if en_preparacion else
            EstadoCola.ALERTA
```

Es una solución simple pero efectiva, ya que no requiere configuraciones adicionales, es fácil de depurar y permite incorporar lógica compleja si se necesita.

4.4.5. Amortiguación y suavizado

Los cambios abruptos entre posiciones pueden hacer que el movimiento de la cola se vea artificial. Para suavizar la transición entre estados y evitar movimientos bruscos, se puede implementar un sistema de interpolación.

Una solución sencilla consiste en tratar la posición de la cola como un "objetivo" *target*, y hacer que la posición real de la cola siga a este objetivo con una velocidad dada. De esta manera, la cola nunca se "teletransportará", sino que transitará suavemente entre estados.

4.5. Sistema Temporal

4.5.1. El problema del tiempo

Para calcular correctamente la posición de la cola en cualquier momento, se requiere un parámetro temporal. Este tiempo debe avanzar de forma constante y ser independiente del *framerate*. Solo así se podrán controlar las velocidades de ondulación de manera coherente y fluida.

4.5.2. Acumulador temporal y *delta*

En Godot, el parámetro **delta** representa el tiempo transcurrido entre dos cuadros consecutivos, lo que permite independencia del *framerate*.

Podemos acumular este valor en cada cuadro para obtener un tiempo total transcurrido. Si queremos que la cola se mueva más rápido, basta con multiplicar **delta**

por un factor. Por ejemplo, podríamos definir un parámetro `VELOCIDAD_ONDULACION` para controlar la rapidez de la animación:

```
cola_tiempo += delta * VELOCIDAD_ONDULACION
```

Este enfoque permite un control directo e intuitivo sobre la velocidad del movimiento de la cola.

4.6. Representación Física

Hasta ahora hemos tratado a la cola como un único punto en el espacio, pero anatómicamente la cola de un gato es una extensión de su columna vertebral. Esto presenta desafíos adicionales:

- ¿Cómo conectamos visualmente la cola al cuerpo del gato?
- ¿Cómo logramos que parezca una extensión natural del esqueleto?

En Godot, esto se resuelve mediante el uso del sistema de esqueletos. Afortunadamente, el motor ya incluye un sistema de *Inverse Kinematics* (IK), que nos permite calcular las posiciones geométricas de varios huesos como si se tratara de una cuerda con un destino.

En el siguiente capítulo abordaremos en detalle cómo configurar los IKs, qué es el sistema de esqueletos en Godot y cómo puede utilizarse para animar la cola del gato. Sin embargo, hasta este punto debería estar claro que necesitamos un punto objetivo hacia el cual la cola debe dirigirse.

Capítulo 5

Sistema de IK y Esqueleto

El sistema de animación de cola que desarrollamos en el capítulo anterior nos proporcionó una solución elegante para simular el comportamiento emocional del gato. Sin embargo, ese sistema funcionaba con un solo punto objetivo: la punta de la cola. En la realidad, la cola de un gato es una extensión continua de su columna vertebral, compuesta por múltiples vértebras que se mueven de forma coordinada.

Del mismo modo, las patas del gato no pueden simplemente aparecer en las posiciones calculadas por nuestro sistema de locomoción procedural. Necesitamos una forma de conectar visualmente estos puntos objetivo con el esqueleto real del gato, creando transiciones suaves que respeten la anatomía del animal.

Aquí es donde entra el sistema de **Inverse Kinematics** (IK) o Cinemática Inversa. Este sistema nos permite calcular automáticamente las posiciones de todos los huesos intermedios cuando conocemos la posición deseada del extremo de una cadena de huesos.

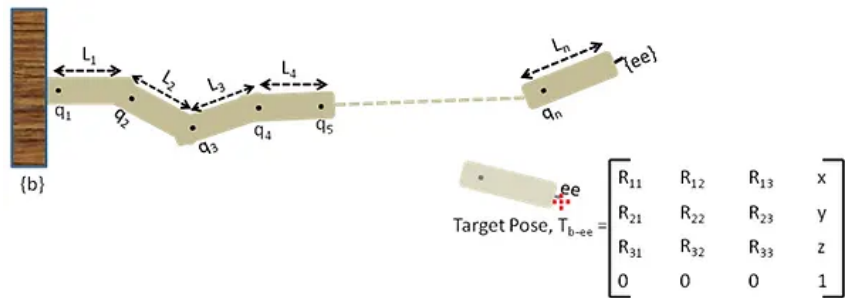


Figura 5.1: Concepto de Inverse Kinematics: dado un objetivo, calcular las articulaciones

La belleza de la cinemática inversa radica en que resuelve automáticamente problemas que serían extremadamente complejos de calcular manualmente. ¿Cómo debe doblarse cada articulación de la pata para que el pie llegue exactamente a la posición deseada? ¿Cómo debe curvarse la columna vertebral para que la cabeza mire hacia un punto específico? El IK responde estas preguntas matemáticamente.

5.1. Configuración de Esqueleto en Godot

Antes de implementar cualquier sistema de IK, necesitamos entender cómo Godot maneja los esqueletos y las jerarquías de huesos. Un esqueleto en Godot es una estructura jerárquica donde cada hueso tiene una posición, rotación y escala relativa a su hueso padre.

5.1.1. Estructura Jerárquica de Huesos

En nuestro gato, el esqueleto sigue una jerarquía natural que refleja la anatomía real. La columna vertebral actúa como el eje central, con huesos que se extienden desde la pelvis hasta la cabeza. Cada pata forma su propia cadena cinemática que se conecta al cuerpo principal.

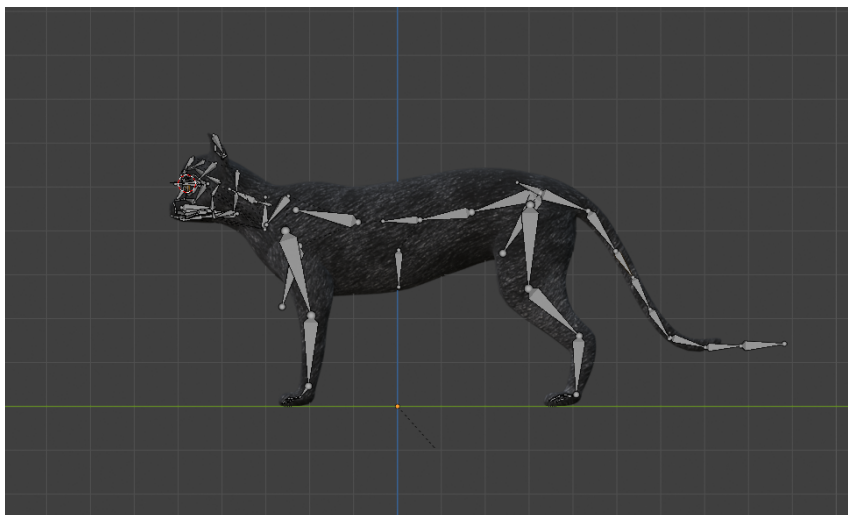


Figura 5.2: Jerarquía de huesos en el esqueleto del gato

La organización que utilizamos incluye huesos de columna nombrados secuencialmente (**spine**, **spine.001**, **spine.002**, etc.) y huesos de extremidades que siguen convenciones anatómicas. Esta nomenclatura coherente es crucial porque nuestro código debe poder encontrar y manipular huesos específicos por nombre.

Los nodos **target** actúan como objetivos para las cadenas de IK. Cada uno de estos nodos (**t_backL**, **t_backR**, **t_frontL**, **t_frontR**, **t_tail**) se posiciona donde queremos que llegue el extremo de cada cadena cinemática. El sistema de IK calcula automáticamente cómo deben rotar todos los huesos intermedios para alcanzar estos objetivos.

5.1.2. Configuración de Nodos IK

Godot proporciona nodos especializados para manejar cinemática inversa. En nuestro sistema, utilizamos múltiples nodos **SkeletonIK3D** que resuelven diferentes cadenas cinemáticas independientemente.

Cada nodo IK requiere configuración específica:

- **Root bone:** El hueso desde donde comienza la cadena IK
- **Tip bone:** El hueso que debe alcanzar el objetivo
- **Target:** El nodo que define la posición objetivo
- **Magnet:** Un punto de atracción que influye en cómo se dobla la cadena

La configuración de múltiples cadenas IK permite control granular sobre diferentes partes del cuerpo. Las patas delanteras y traseras pueden moverse independientemente, mientras que la columna vertebral responde a la posición deseada de la cabeza.

5.2. El Problema de la Columna Vertebral

Las patas del gato se resuelven relativamente bien con IK tradicional, pero la columna vertebral presenta desafíos únicos. Una columna vertebral real no es una cadena rígida de articulaciones; es más parecida a una cuerda flexible que debe mantener ciertas proporciones mientras se adapta a la forma deseada.

5.2.1. Limitaciones del IK Tradicional

El IK tradicional funciona bien para extremidades con articulaciones definidas, pero la columna vertebral requiere un enfoque diferente. Si simplemente aplicáramos IK desde la pelvis hasta la cabeza, obtendríamos movimientos antinaturales donde algunas vértebras se comprimen excesivamente mientras otras se estiran de forma poco realista.

Esta observación sugiere que podemos pensar en la columna vertebral como un problema de distribución: necesitamos distribuir la longitud total de la columna de manera uniforme entre todas las vértebras, mientras dirigimos la cabeza hacia el objetivo deseado.

5.2.2. El Algoritmo de Cuerda

Nuestro enfoque trata la columna vertebral como una cuerda con segmentos de longitud fija. El algoritmo funciona en dos pasos principales:

Primer paso - Establecer el objetivo: Colocamos el extremo de la cuerda” (la cabeza) en la posición deseada.

Segundo paso - Propagar hacia atrás: Iteramos desde la cabeza hacia la pelvis, posicionando cada vértebra a la distancia correcta de la siguiente, manteniendo la longitud original entre vértebras adyacentes.

```
func get_solution_column(target: Vector3, puntos: Array) -> Array:
    var new_puntos := []
    new_puntos.resize(puntos.size())
    new_puntos[puntos.size() - 1] = target

    for i in range(puntos.size() - 2, -1, -1):
```

```
var dir = target - puntos[i]
var dir_u = dir.normalized()
var distancia = puntos[i + 1].distance_to(puntos[i])
new_puntos[i] = new_puntos[i + 1] + dir_u * -1.0 * distancia
target = new_puntos[i]

return new_puntos
```

Este algoritmo garantiza que la distancia entre vértebras adyacentes se mantenga constante, evitando compresiones o estiramientos antinaturales.

5.3. Sistema de Amortiguación con Resortes

Simplemente posicionar la cabeza en la ubicación calculada por nuestro sistema de locomoción resultaría en movimientos demasiado bruscos y poco naturales. Los gatos reales no mueven la cabeza instantáneamente; hay inercia, elasticidad y amortiguación en el movimiento.

5.3.1. Física de Resortes

Implementamos un sistema de resortes físicos que trata la cabeza como una masa conectada por un resorte a la posición objetivo. La ecuación básica del resorte es:

$$F = -kx - bv$$

donde:

- F es la fuerza aplicada
- k es la rigidez del resorte (`spring_stiffness`)
- x es el desplazamiento desde la posición objetivo
- b es el coeficiente de amortiguación (`spring_damping`)
- v es la velocidad actual

Esta ecuación produce movimiento que se siente orgánico: la cabeza acelera hacia el objetivo cuando está lejos, pero se desacelera gradualmente cuando se acerca, evitando oscilaciones excesivas.

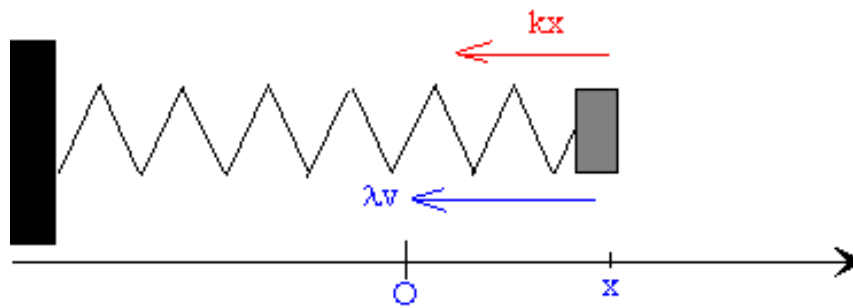


Figura 5.3: Respuesta de un sistema de resorte amortiguado

5.3.2. Integración de Verlet

Para simular la física del resorte, utilizamos integración de Verlet, que es numéricamente estable y conserva bien la energía:

```
# Calculate acceleration (F = ma, so a = F/m)
var acceleration = spring_force / mass

# Update velocity and position using verlet integration
velocity += acceleration * delta
current_head_pos += velocity * delta
```

Este enfoque permite que la cabeza responda de forma natural a los cambios en el objetivo, creando movimiento fluido y realista.

5.4. Cálculo de Orientación de Vértebra

Una vez que tenemos las posiciones de todas las vértebras, necesitamos calcular cómo debe rotar cada hueso para que el esqueleto se vea natural. Esto involucra tanto rotación en el plano horizontal (dirección hacia donde apunta la columna) como rotación vertical (curvatura hacia arriba o abajo).

5.4.1. Proyección en Plano Horizontal

Para cada vértebra, calculamos primero su orientación en el plano horizontal. Esto determina hacia qué dirección "mira" cada segmento de la columna:

```
var dir = positions[i+1] - positions[i]
var dir_proj_zx = Vector3(dir.x, 0, dir.z)
pos.basis = basis_pos[i].basis * Basis(Vector3.BACK,
    Vector3(0,0,10).signed_angle_to(dir_proj_zx, Vector3.DOWN))
```

Esta operación rota cada vértebra para que apunte hacia la siguiente vértebra en la cadena, pero solo en el plano horizontal.

5.4.2. Rotación Vertical

La rotación vertical maneja la curvatura de la columna hacia arriba o abajo. Utilizamos el vector lateral de cada vértebra como eje de rotación:

```
var vec_plane = pos.basis.x
var dir_proj_vec_plane = dir - dir.dot(vec_plane.normalized()) * vec_plane
    .normalized()
var front_proj_vec_plane = Vector3(dir_proj_vec_plane.x, 0,
    dir_proj_vec_plane.z)

pos.basis *= Basis(basis_pos[i].basis.x.normalized(),
    -dir_proj_vec_plane.signed_angle_to(front_proj_vec_plane, pos.basis.x))
```

Esta operación adicional permite que la columna se curve naturalmente hacia arriba cuando el gato mira hacia objetos elevados, o hacia abajo cuando explora el suelo.

5.5. Coordinación de Sistemas IK

Tener múltiples sistemas IK trabajando simultáneamente requiere coordinación cuidadosa para evitar conflictos y asegurar movimiento coherente.

5.5.1. Actualización de Posiciones Target

El sistema de coordinación actualiza constantemente las posiciones de los nodos target basándose en los cálculos del sistema de locomoción:

```
func update_targets_pos(delta):
    var patas = objetivo.call("obtener_patas")
    var cola_objetivo = objetivo.call("obtener_posicion_cola")

    t_backL.global_position = patas["backL"]
    t_backR.global_position = patas["backR"]
    t_frontL.global_position = patas["frontL"]
    t_frontR.global_position = patas["frontR"]

    # Cola se mueve suavemente con lerp
    var factor_lerp = VELOCIDAD_SUAVIZADO_COLA * delta
    t_tail.global_position = t_tail.global_position.lerp(cola_objetivo,
        factor_lerp)
```

Nota la diferencia en el tratamiento: las patas se mueven instantáneamente porque nuestro sistema de locomoción ya maneja la interpolación suave entre posiciones. La cola, sin embargo, se mueve gradualmente para evitar movimientos bruscos.

5.5.2. Sistema de Magnets

Los "magnets.^{en} Godot IK son puntos de atracción que influyen en cómo se doblan las cadenas cinemáticas. Configuramos estos magnets para crear posturas

naturales:

```
func update_magnet_pos():
    ik_backL.magnet = t_frontL.position
    var vec = t_frontL.position - t_backL.position
    vec = vec.normalized()
    ik_backL2.magnet = t_backL.position - vec*100

    ik_backR.magnet = t_frontR.position
    vec = t_frontR.position - t_backR.position
    vec = vec.normalized()
    ik_backR2.magnet = t_backR.position - vec*100

    ik_frontL.magnet = t_backL.position
    ik_frontR.magnet = t_backR.position
```

Esta configuración crea una "atracción cruzada" donde las patas traseras se sienten atraídas hacia las delanteras del lado opuesto, y viceversa. Esto simula la postura natural de los cuadrúpedos donde las patas se posicionan para maximizar la estabilidad.

5.6. Integración con Transformaciones del Cuerpo

El sistema IK no puede funcionar en aislamiento; debe integrarse con el sistema de posicionamiento global del gato.

5.6.1. Posicionamiento y Orientación del Cuerpo

El cuerpo del gato se posiciona basándose en el centro calculado por el sistema de locomoción, con un offset vertical para acomodar la altura del modelo:

```
func update_root_pos_rot():
    var center = objetivo.call("obtener_centro")
    center.y = center.y - offset_y
    global_transform.origin = center

    var dir = objetivo.call("obtener_direccion")
    var angle_y = atan2(dir.x, dir.z)
    global_transform.basis = Basis(Vector3.UP, angle_y)
```

Esta función sincroniza la orientación del modelo 3D con la dirección calculada por el sistema de locomoción, asegurando que el gato siempre mire hacia donde se está moviendo.

5.6.2. Cálculo de Posición de Cabeza

La posición objetivo de la cabeza se calcula considerando múltiples factores:

```
func get_target_pos():
    var patas = objetivo.call("obtener_patas")
    if (patas):
        var out = (patas["frontL"]/scale.x + patas["frontR"]/scale.x)/2
        out.y += offset_y/2
        return out + objetivo.call("obtener_direccion")*2.5
    return Vector3(0,0,0)
```

Esta función posiciona la cabeza ligeramente por delante del centro de las patas delanteras, en la dirección de movimiento. Esto crea la postura natural donde el gato mira hacia donde se dirige.

5.7. Consideraciones de Rendimiento

El sistema IK puede ser computacionalmente costoso, especialmente cuando se ejecuta para múltiples cadenas cinemáticas en cada frame.

5.7.1. Optimización de Cálculos

Varias estrategias pueden mejorar el rendimiento:

- **Limitar iteraciones:** Los algoritmos IK iterativos pueden limitarse a un número máximo de iteraciones por frame
- **Caching de resultados:** Resultados de cálculos costosos pueden almacenarse cuando las condiciones no han cambiado significativamente
- **Actualización selectiva:** No todas las cadenas IK necesitan actualizarse en cada frame si el cambio es mínimo

5.7.2. Configuración de Frecuencia

Los diferentes sistemas pueden ejecutarse a diferentes frecuencias según su importancia visual:

- **Patas:** Máxima frecuencia (cada frame) para mantener contacto preciso con el suelo
- **Columna vertebral:** Frecuencia media, ya que cambios graduales son menos perceptibles
- **Cola:** Frecuencia reducida, ya que el movimiento natural de la cola es relativamente lento

El sistema resultante crea una animación fluida y natural que responde dinámicamente al entorno, manteniendo la anatomía correcta del gato mientras permite el rango completo de movimientos necesarios para navegar por la ciudad procedural.

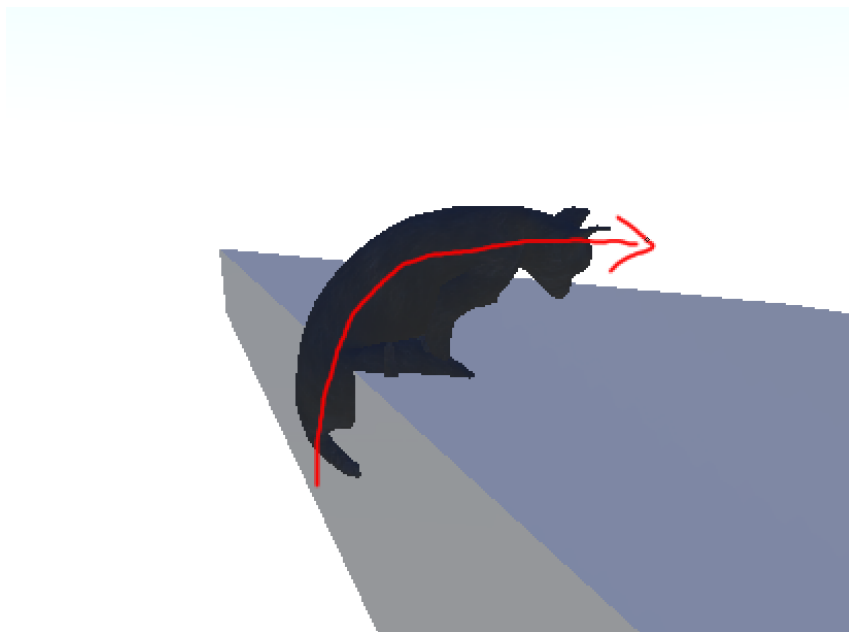


Figura 5.4: Sistema IK en acción: manteniendo anatomía natural en diferentes posturas

Parte IV

Generación de Ciudad Procedural

Capítulo 6

Sistema de Chunks Espaciales

6.1. Mundos Procedurales: Minecraft

En este capítulo abordaremos en detalle cómo generar una ciudad procedural, explicando inicialmente qué es un espacio procedural.

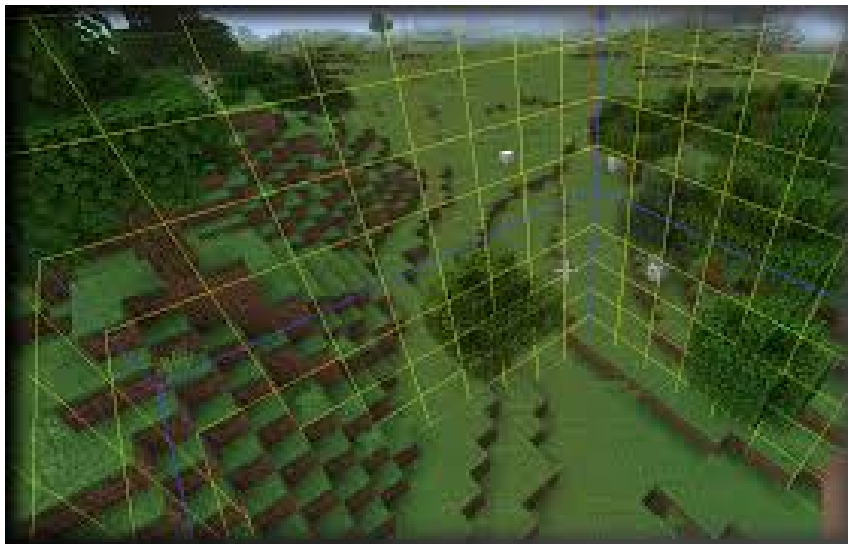


Figura 6.1: Minecraft: Chunks

Minecraft constituye el mejor ejemplo para ilustrar un sistema de generación y renderización de mundos procedurales. El problema fundamental es el siguiente: **¿Cómo podemos renderizar un mundo infinito?** La respuesta inmediata es: no podemos.

Por tanto, la solución siempre implicará alguna estrategia para renderizar únicamente la porción del mundo que sea visible para el usuario. Esto nos lleva a una pregunta más fundamental: **¿qué define al mundo?**

Si consideráramos un modelo 3D o escena 3D tradicional (como en Blender), el mundo estaría completamente definido a priori. Es decir, tendríamos una malla que representa la totalidad del mundo.

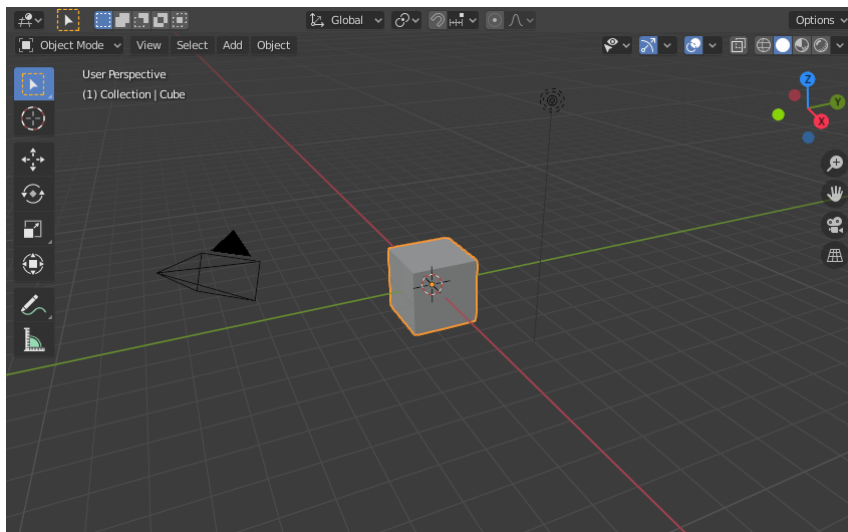


Figura 6.2: Blender: Escena con Cubo

Sin embargo, en casos de generación procedural, queremos que el mundo se genere dinámicamente a medida que el usuario avanza. Por tanto, el mundo no puede estar predefinido, ya que su naturaleza infinita requeriría un espacio de almacenamiento infinito.

6.2. Partición Espacial del Mundo

La solución a este problema radica en la **partición del mundo**. La estrategia empleada por Minecraft, y adoptada también en nuestro sistema, consiste en particionar el mundo en **chunks**. Un chunk es una región cuadrada del mundo vista desde una perspectiva 2D superior. Esencialmente, constituye una grilla que representa la división de un mundo conceptualmente infinito.

Es trivial clasificar cualquier punto dado a un chunk determinado mediante sus coordenadas. Esta lógica será fundamental para nuestro sistema de renderizado.

6.2.1. Cálculo de Coordenadas de Chunk

Para determinar a qué chunk pertenece un punto en el mundo, empleamos una división entera simple:

```
func get_chunk_coord(world_x: float, world_y: float, chunk_size: int) ->
    Vector2i:
    return Vector2i(
        floor(world_x / chunk_size),
        floor(world_y / chunk_size)
    )
```

Esta función transforma cualquier coordenada mundial en coordenadas de chunk, permitiendo así asignar posiciones específicas a cada chunk. Conceptualmente, es equivalente a agrupar el mundo en unidades más grandes o dibujar una cuadrícula de mayor escala.

6.2.2. El Mundo como una Función

Una vez establecida la división del mundo en chunks, surge la pregunta: **¿qué es el mundo?** La respuesta es conceptualmente simple: **el mundo es una función**. Aunque no necesariamente en el sentido matemático estricto, se comporta como tal al tener un conjunto o malla definida para cada punto o espacio, similar a una superficie explícita.

En nuestro contexto, el mundo posee la característica de permitir **consultas espaciales**. Es decir, si deseamos conocer el contenido del chunk (132,45), la "función" del mundo puede retornar la malla correspondiente a esa región.

La función del mundo constituye, por tanto, el núcleo de la generación procedural, permitiendo renderizar únicamente los chunks próximos al usuario. Nuestro objetivo es implementar este patrón de manera que, dado un bounding box, podamos retornar el mundo correspondiente a ese espacio.

6.3. Generación de Ciudades

Este marco conceptual nos introduce a la generación de ciudades. **¿Podemos aplicar la misma estrategia? ¿Cómo generamos una ciudad?**

Antes de plantear el algoritmo generativo, debemos asegurar el cumplimiento de las normas de generación procedural: debemos ser capaces de, dada una región, retornar su contenido correspondiente. Este contenido debe ser **determinístico**, es decir, invariable ante múltiples visitas del usuario a la misma ubicación.

6.3.1. El Algoritmo de Generación

Proponemos un algoritmo de generación de ciudades intuitivo cuya primera intención es simular distritos y manzanas. Las agrupaciones humanas siguen este patrón natural. En lugar de dividir el mundo en calles y avenidas (como otros algoritmos), nuestra aproximación se centra en generar manzanas y distritos.

Empleamos el **algoritmo de Voronoi**, capaz de generar figuras similares a manzanas o distritos, desde formas cuadradas hasta polígonos complejos.

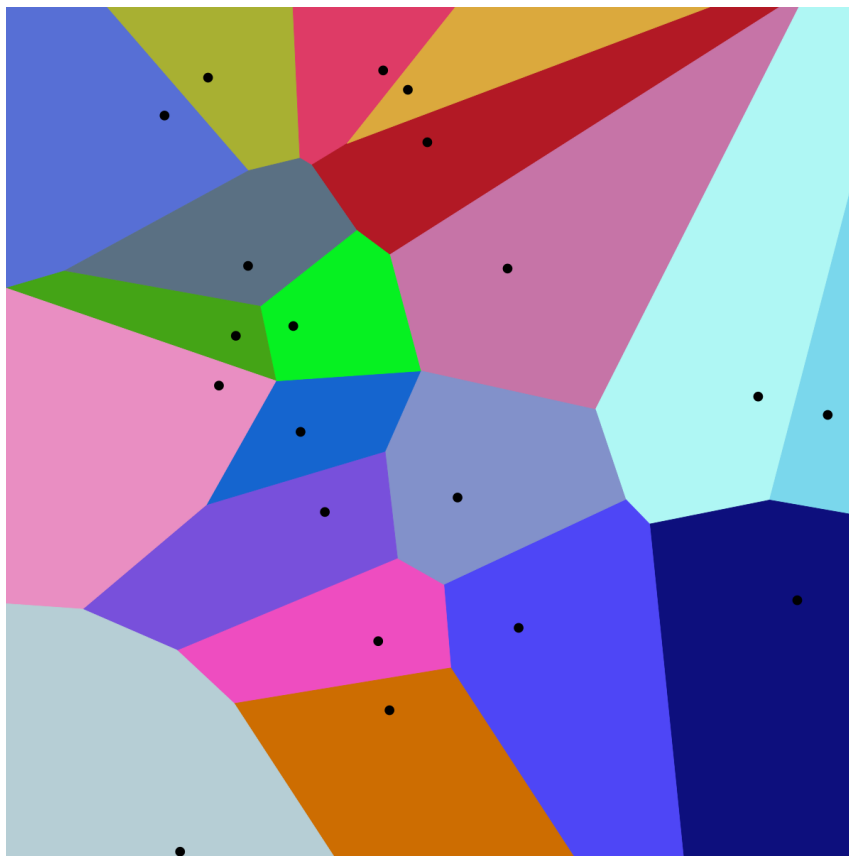


Figura 6.3: Diagrama de Voronoi

Sin embargo, Voronoi requiere puntos como entrada. No opera como una función procedural directa, sino que necesita puntos para ser calculado. Necesitamos, por tanto, una función procedural que retorne puntos en una región dada.

6.3.2. Hash Determinístico con Semilla

La generación determinística de puntos se vuelve crucial para la generación del mundo. Esta etapa no admite errores, ya que constituye la base del algoritmo de generación de ciudades.

El proceso es conceptualmente simple. Una opción consiste en iterar sobre una grilla interna y determinar mediante un generador aleatorio si se dibujará o no el punto. Sin embargo, un generador aleatorio típico no es determinístico. La solución radica en que la mayoría de lenguajes permiten asignar una **semilla** al generador aleatorio. Esta semilla define la secuencia de valores, garantizando determinismo. Al proporcionar al generador una semilla creada con las coordenadas del chunk, aseguramos que para un chunk dado siempre obtenemos los mismos puntos.

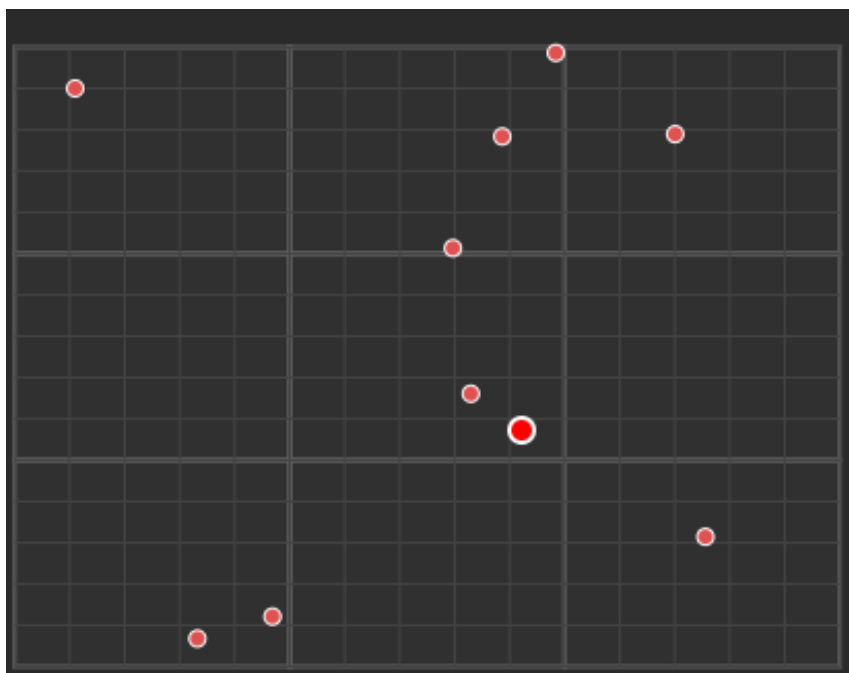


Figura 6.4: Implementación básica del sistema de consulta de puntos para 9 chunks

6.3.3. Voronoi: Generación de Ciudades

¿Es tan simple el proceso? ¿Podemos aplicar Voronoi sobre los puntos generados y obtener una ciudad completa? Teóricamente sí, aunque existen problemas que debemos abordar.

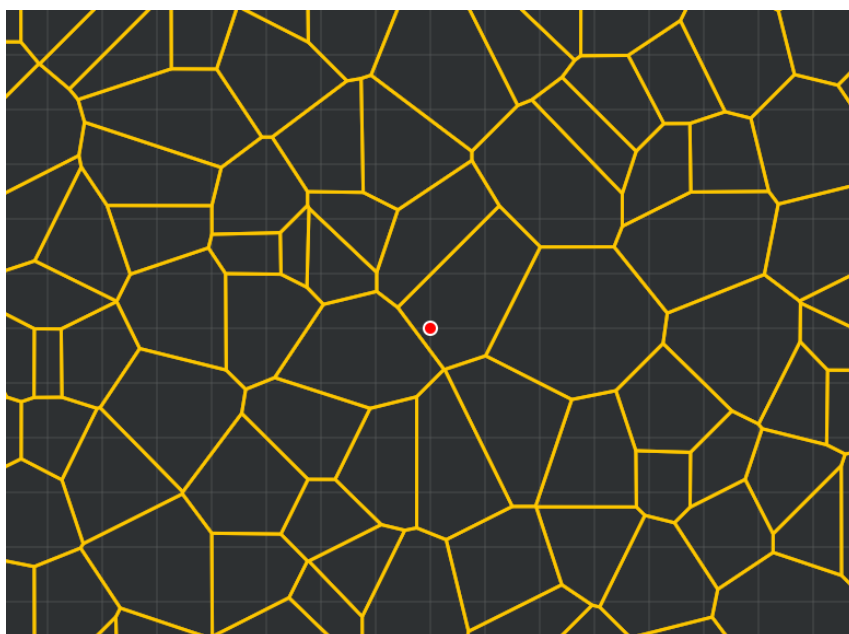


Figura 6.5: Diagrama de Voronoi Generado — HTML

La figura muestra el mismo sistema anterior con Voronoi aplicado y visualizado.

¿Es esto suficiente? La respuesta es **NO**.

Si modificamos la distancia de renderización de chunks (cuántos chunks renderizamos), encontramos un problema significativo:

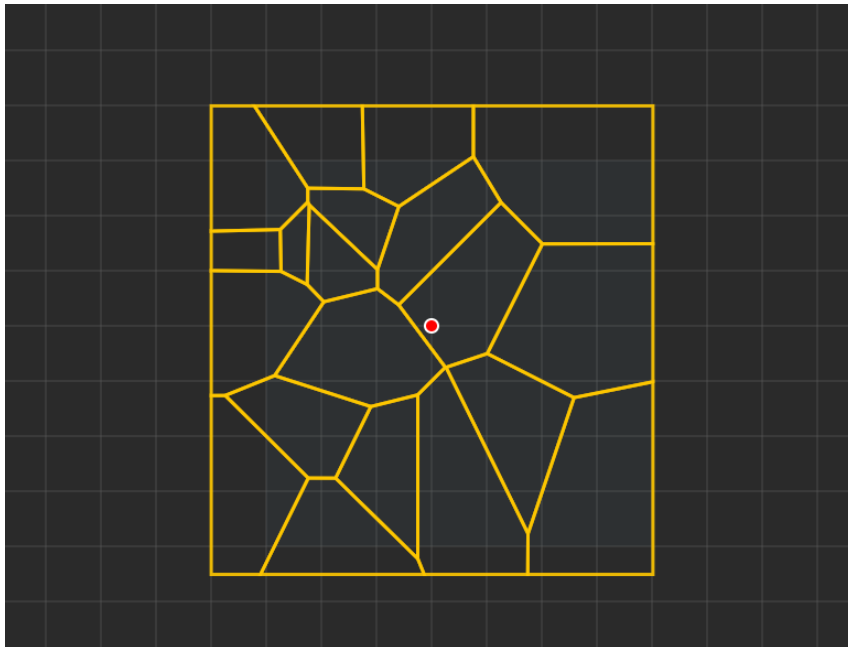


Figura 6.6: Problema de Chunks

La imagen muestra el mismo mundo con idéntica semilla pero con menor distancia de renderizado. Observamos que **los polígonos cerca del borde NO SON ESTABLES**.

Este constituye un problema crítico: habíamos garantizado la estabilidad de los puntos, pero los polígonos no lo son, significando que los distritos cambian a medida que avanza el usuario.

¿Qué distritos son verdaderamente estables?

6.4. Sistema de Estabilidad de Chunks

Definamos el **problema de estabilidad de chunks**. Este problema determinará qué chunks pueden garantizar la existencia de al menos un punto. Sabemos que este punto generará un polígono (región de puntos más cercanos, básicamente Voronoi). La pregunta es: ¿el polígono representa el mundo real.º solo una deformación de este?

Comenzamos buscando una definición para lo que constituye un **chunk estable**. Si tenemos un punto en un chunk, ¿podemos garantizar que la región generada por ese punto pertenecerá al mundo original?

La respuesta completa es: **depende**. Depende de si existen puntos más cercanos fuera del área visible por el jugador. Sin embargo, la garantía de existencia de un punto por chunk sugiere la siguiente afirmación: **Solo los chunks de borde son**

inestables, es decir, únicamente los puntos dentro de algún chunk de borde no generan regiones estables.

La demostración se basa en que si existiese un punto fuera de la región visible que fuera el más cercano a un chunk rodeado por otros 8 chunks (cada uno con al menos un punto), entonces se puede deducir que el punto más cercano realmente era uno de esos 8 y no el indicado, lo cual constituye una contradicción.

6.4.1. Algoritmo de Generación

Definamos formalmente nuestro algoritmo de generación. La idea consiste en, dada una región, generar los puntos que generen distritos (denominados **puntos avenida**) y generar manzanas (**puntos calle**).

El algoritmo presenta dos capas jerárquicas: primero los distritos y posteriormente las manzanas. Las manzanas constituyen espacios dentro de un distrito, por lo que sus polígonos serán recortados para no exceder los límites del distrito.

Con este enfoque, podemos implementar un **ChunkManager** que maneje la jerarquía y proporcione la lógica para generar las mallas.

Si desarrollamos un método para dividir las manzanas en lotes, habremos completado la generación de ciudades. El algoritmo implementado produciría un resultado como el siguiente:



Figura 6.7: Ciudad Generada en HTML

En la imagen observamos el algoritmo en pleno funcionamiento, evidenciando la generación de distritos y manzanas. Para la visualización de calles, aplicamos un padding simple en las manzanas antes de dividir las en lotes. Este padding produce el efecto de pistas y avenidas.

Con esto, disponemos de la información suficiente para generar las mallas de cada construcción, conociendo su ubicación exacta. Sin embargo, surge la pregunta: **¿debemos recalculamos todas las mallas cada vez que nos movemos?**

6.4.2. Sistema de Caché Básico

La respuesta a la pregunta anterior es **NO**. No es necesario recalculamos constantemente; de hecho, solo es necesario calcular las mallas de los polígonos que anteriormente eran inestables y ya no lo son.

Para una implementación simple, definiremos un chunk estable por su distancia al jugador. Si el chunk está dentro de cierto radio (siempre que no sea uno de borde), entonces es estable y podemos generar las mallas. Las coordenadas enteras de los chunks permiten usarlas como claves para un caché clave-valor donde almacenar las mallas generadas y el polígono de cada manzana.

La implementación del caché es directa:

```
static var stability_cache: Dictionary = {}

func is_cache_valid(cache_key: String) -> bool:
    if not stability_cache.has(cache_key):
        return false

    var entry = stability_cache[cache_key]
    return true
```

Al generar manzanas, siempre requerimos guardarlas en el caché si son estables. Si no son estables, simplemente no generaremos la malla, asegurando un uso eficiente de nuestros recursos.

En la siguiente sección explicaremos detalladamente cómo generar la malla dado un polígono.

Capítulo 7

Sistema de generación procedural de casas

En este punto lo que hace falta es poder crear casas en el terreno, dado que la ciudad podrá ir creciendo 'infinitamente' es necesario poder crear las mallas a tiempo real. Puesto que una casa no es una forma conocida como tal, tenemos que hacerla desde cero, la ventaja que tenemos es que las casas son, al fin y al cabo, figuras simples. Para esto hemos ideado un sistema de generación procedural cuyo objetivo es crear mallas tridimensionales de viviendas, a partir de parámetros generados aleatoriamente mediante una semilla. Este enfoque garantiza la *determinismo*: si se emplea la misma semilla, se obtiene siempre la misma secuencia de casas. Esta característica resulta útil para mantener coherencia entre sesiones o replicar escenarios exactos durante pruebas y desarrollo.

La inspiración para el diseño de cada vivienda proviene de las construcciones típicas del Cusco. Estas casas, caracterizadas por sus techos inclinados de tejas, ofrecen un entorno ideal para explorar dinámicas de movimiento, salto y navegación del gato protagonista dentro de la escena.

Desde el punto de vista técnico, el sistema acepta como entrada un polígono base —la huella de la casa— y, junto a los parámetros aleatorios, define la volumetría y nivel de detalle de la construcción. El resultado final es una malla 3D que puede ser instanciada directamente en el motor Godot, tanto como estructura principal como con elementos decorativos adicionales.

Para facilitar la comprensión del proceso completo, se organiza en dos bloques principales:

- **Bloque I – Construcción geométrica abstracta:** Se encarga de generar la casa a nivel estructural, produciendo una lista de vértices y caras que definen su forma tridimensional. Aquí se aplican reglas consistentes para definir paredes, techos a dos aguas, buhardillas y ajustes al terreno usando ruido perlin.
- **Bloque II – Representación en Godot:** Toma la geometría generada y la transforma en nodos válidos del motor. Este módulo crea las mallas, les asigna materiales, gestiona su orientación espacial y añade elementos decorativos como ventanas y marcos.

Esta división entre lógica generativa y representación visual permite modularidad, facilita el mantenimiento del sistema y abre la posibilidad de reutilizarlo en otros contextos del proyecto.

7.1. Bloque I: Construcción geométrica de una casa

7.1.1. Representación geométrica abstracta

Polígono base. La base de la casa es un polígono simple en el plano XZ , representado como una lista ordenada de puntos tridimensionales (**Vector3**) con coordenada $y = 0$. Este polígono define la huella sobre la que se construirá el volumen de la casa.

Vértices. A lo largo del proceso, se van agregando puntos a una lista global de vértices, todos representados como instancias de **Vector3**. Esta lista mantiene el orden en que se agregan los vértices, lo cual es esencial para la construcción posterior de las caras.

Caras. Cada cara se representa como una lista con el siguiente formato:

$$[\text{tipo}, n, \text{idx}_1, \text{idx}_2, \dots, \text{idx}_n]$$

donde **tipo** es un entero que indica el propósito de la cara (por ejemplo: 0 para techo, 1 para pared, 2 para base), **n** indica el número de vértices que forman la cara, y **idx_i** son los índices en la lista global de vértices.

Independencia de motor. Esta representación permite mantener la lógica de construcción separada de la lógica de visualización, facilitando pruebas, depuración y reutilización del sistema en otros contextos o motores gráficos.

7.1.2. Etapas de construcción

En la figura 7.1 se describe la estructura resumida de las etapas de construcción de una casa. Detallemos un poco cada parte, el desnivel de las casas se hace a partir de ruido perlin para darle más realismo a cada edificación. Los otros ajustes del polígono se hacen con el fin de priorizar obtener cuadriláteros en vez de pentágonos u otros polígonos de más lados que serían casos atípicos en la vida real.

En general todo el proceso, es muy fácil, solo es agregar los puntos que conforman las caras de mi casa. Según el tipo de techo que toque para la casa actual se hace el diseño correspondiente.

7.1.3. Incorporación de aleros

Hasta este punto tenemos una casa con paredes y un techo, lo cual es suficiente desde el punto de vista estructural. Sin embargo, desde el punto de vista estético

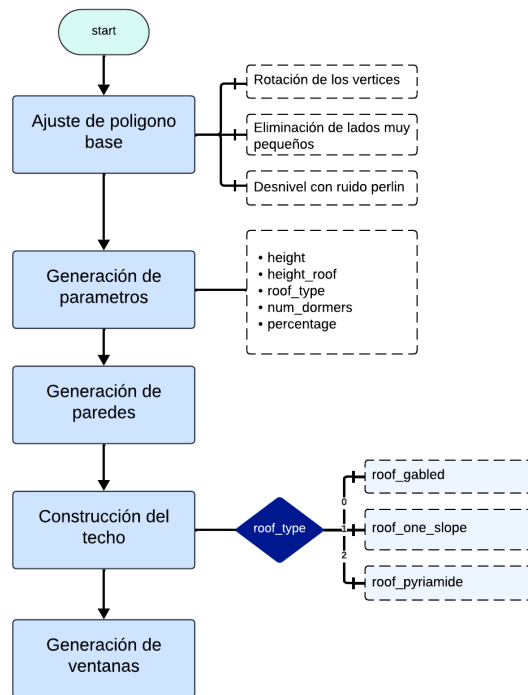


Figura 7.1: Etapas de construcción

(y también realista), resulta evidente que algo falta: en las casas reales con tejados, las cubiertas suelen sobresalir ligeramente del borde de las paredes. Este borde sobresaliente se conoce como *alero*.

Para lograr este efecto, no basta con usar directamente los puntos del techo que están alineados con las paredes. En su lugar, debemos calcular nuevos puntos desplazados hacia afuera. Supongamos que los puntos P_1 y P_2 son los bordes del techo alineados con las paredes de la casa (ver figura 7.2). Para generar el alero, reemplazaremos estos puntos por nuevas versiones desplazadas:

$$P' = P - \hat{u} \cdot s - \hat{v} \cdot s$$

Donde: - s es el tamaño del alero (`eave_size`), - $\hat{u}$ es un vector "horizontal" (unitario), que sale desde cada P y va a su punto contrario. - $\hat{v}$ es un vector "vertical" (unitario), que apunta hacia su respectiva cresta (parte más alta del tejado) desde el punto.

Este desplazamiento se aplica a cada uno de los puntos que conforman una cara del techo, con una consideración: el componente v del desplazamiento para los puntos pertenecientes a la cumbrera, es decir (P_3 y P_4), es cero.

En código, este cálculo se realiza dentro de la función `get_new_eave()`, que devuelve una versión desplazada del punto original según el tipo de borde y si pertenece a una buhardilla o no.

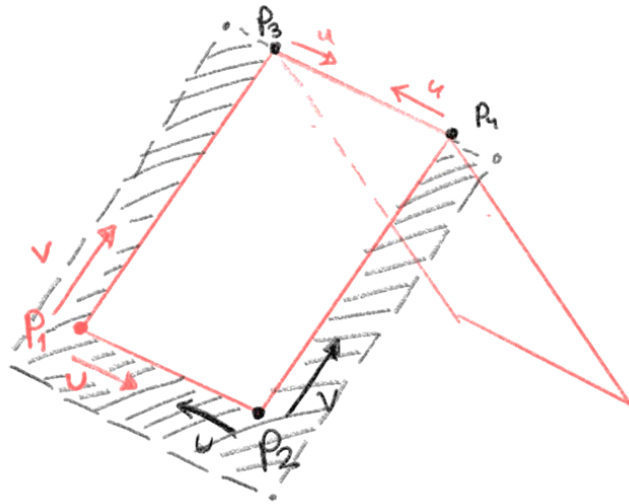


Figura 7.2: Expansión de una cara del techo para formar aleros.

7.1.4. Lógica para la creación de buhardillas

Actualmente el diseño de la casa va bien, pero sigue siendo un diseño muy plano, con pocos lugares a los que saltar. Para solucionar esto, crearemos buhardillas, que son pequeñas estructuras que sobresalen del tejado principal y aportan tanto funcionalidad como estética a la casa.

Para generar una buhardilla nos basaremos en dos puntos A y B que pertenecen a la pared y al plano inclinado del techo sobre el cual se quiere crear la buhardilla. Además, tenemos una altura H que es el mínimo entre la altura del plano inclinado y la distancia entre A y B , con el objetivo de formar una buhardilla con un triángulo bien proporcionado en la mayoría de casos.

Observemos la figura 7.3. El punto R , que es la cumbrera de la buhardilla, se calcula a partir de la fórmula:

$$R = \frac{A + B}{2} + \text{Vector3.UP} \times H$$

Este punto define la parte más alta de la buhardilla. Sin embargo, para que encaje correctamente con el plano del techo principal, se necesita otro punto, al que llamaremos p_ridge , que se obtiene proyectando R sobre dicho plano en dirección del vector $depth$. Esta intersección garantiza que la buhardilla se acople suavemente al techo.

Una vez obtenidos los puntos A , B , R y p_ridge , se agregan a la lista global de vértices y se crean tres caras: los dos laterales triangulares de la buhardilla y su cara frontal (ver figura 7.3).

La función `generate_n_dormers()` se encarga de distribuir n buhardillas de manera uniforme sobre un segmento del techo. El espacio se divide de tal manera que haya una separación entre cada buhardilla, y se calcula un tamaño efectivo basado en el porcentaje deseado de ocupación del espacio total.

En resumen, la generación de buhardillas implica:

- Calcular la posición de cada buhardilla dentro del segmento del techo.

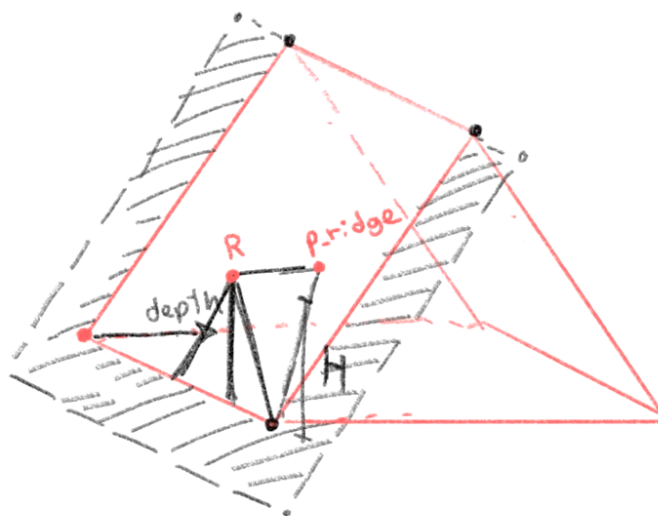


Figura 7.3: Estructura de una buhardilla.

- Determinar su base (A y B), su altura H (limitada por el techo original) y su cumbrera.
- Proyectar correctamente la cumbrera sobre el plano inclinado.
- Construir las tres caras de la estructura (dos laterales y un frente).
- Opcionalmente, extender los bordes para agregar aleros.

7.1.5. Incorporación de ventanas

Hasta este punto la generación de casas va bien, como un plus podemos agregar ventanas, que a su vez nos sirven como fuente de iluminación en escenas de noche. La idea es insertar una ventana dentro de la cara frontal, respetando sus dimensiones y orientación.

El proceso es bastante simple: lo primero es obtener los tres puntos que definen la cara frontal, normalmente un triángulo formado por los puntos A , B y la cumbrera. A partir de estos puntos, se calcula el centro del triángulo, que servirá como punto central de la ventana.

Luego, se calcula la **normal** de la cara para saber en qué dirección debe mirar la ventana. A partir de esta normal, se construye un sistema de coordenadas local: un vector **right** que es perpendicular a la normal, y un vector **up** que completa el sistema ortonormal.

Con este sistema de referencia, se proyectan todos los puntos del triángulo para obtener su ancho y alto en coordenadas locales. Esto permite determinar el tamaño máximo que puede tener la ventana sin salirse del polígono. Para mayor seguridad, se aplica un *factor de escala*, que reduce ligeramente el tamaño de la ventana para que encaje con holgura.

Finalmente, se crea el nodo de la ventana con sus marcos y cristales. El diseño está dividido en dos paneles verticales separados por un marco central, y rodeado por marcos superior, inferior, izquierdo y derecho. Todo esto se construye en función de la anchura y altura deseadas, posicionando cada componente usando los vectores `right`, `up` y la normal calculada.

7.2. Bloque II: Representación en Godot

En esta etapa transformamos la geometría abstracta de la casa —producida en forma de vértices y caras en el Bloque I— en una representación visual concreta dentro del motor Godot. Esto se logra mediante la construcción de una malla optimizada y compatible con el motor gráfico, la cual puede ser instanciada como un nodo visible en la escena.

7.2.1. Conversión de geometría a malla

La geometría generada (listas de vértices y caras) se convierte en una malla utilizando un objeto `ArrayMesh`. Este recurso de Godot permite definir manualmente la estructura de una malla a través de arreglos de datos como posiciones de vértices, índices de triángulo, coordenadas UV, normales y tangentes. Es especialmente útil para generar geometría en tiempo de ejecución, como en este caso.

Un aspecto importante de `ArrayMesh` es que **solo admite triángulos** como primitivas. Por ello, cualquier cara cuadrilateral debe dividirse en dos triángulos antes de ser enviada al arreglo de índices. Esta conversión se hace de forma consistente para mantener la orientación y coherencia visual de las superficies.

7.2.2. Texturizado y proyección UV

Para cada cara se realiza una proyección a coordenadas UV, con el fin de aplicar texturas correctamente sobre las superficies. Esta proyección se basa en un sistema de referencia local definido por tres vectores: un punto de origen, una tangente y una bitangente. Estas direcciones se calculan en base a la geometría de la cara, de forma que la textura se oriente correctamente sobre paredes o techos, sin necesidad de editar UVs manualmente.

El escalado de las coordenadas UV se ajusta dependiendo del tipo de superficie: se aplica un `wall_uv_scale` para paredes y un `roof_uv_scale` para techos. Esto permite controlar cuántas veces se repite la textura según la extensión del objeto.

7.2.3. Cálculo de tangentes

Los efectos de iluminación avanzados (como *normal mapping*) requieren información sobre la tangente y bitangente de cada triángulo. Por ello, para cada grupo de tres vértices se calcula un vector tangente realista usando las derivadas entre coordenadas UV y posiciones espaciales. Esto garantiza que la iluminación responda correctamente en cada punto del modelo, incluso si la textura está rotada o escalada.

Además, se determina la orientación de la tangente usando el producto cruzado con la normal y un valor de **handedness** que asegura consistencia visual.

7.2.4. Separación por materiales

Las caras se dividen en dos grandes grupos: techos y otras superficies (paredes y piso). Esta clasificación se codifica mediante el primer elemento de cada entrada en la lista de caras. Cada grupo se procesa por separado y se añade como una **superficie distinta** al `ArrayMesh`, lo que permite asignar materiales específicos a cada tipo de superficie.

Esto permite, por ejemplo, usar una textura diferente para los techos que para las paredes, sin necesidad de múltiples nodos o mallas.

7.2.5. Creación del nodo visual final

Una vez agregadas todas las caras, se encapsula la malla dentro de un nodo `MeshInstance3D`, que puede ser insertado directamente en la escena. Este nodo contiene toda la geometría, los UVs, las tangentes y los materiales, y es el encargado de renderizar la casa completa en pantalla.

Gracias a esta estructura modular, es posible añadir subelementos decorativos (como ventanas o marcos) como hijos del nodo, sin alterar la integridad de la malla base.

Parte V

Sistema de Interfaz y Visualización

Capítulo 8

Sistema de Cámara Dinámico

El sistema de cámara representa uno de los desafíos más complejos en el desarrollo de juegos 3D. Una cámara mal implementada puede arruinar completamente la experiencia del usuario, sin importar qué tan buenos sean los otros sistemas del juego. En nuestro proyecto, necesitamos una cámara que pueda seguir a un gato ágil mientras salta entre techos, mantener una vista clara del entorno urbano, y responder intuitivamente a los controles del usuario.

Pero antes de escribir una sola línea de código, debemos preguntarnos: **¿qué hace que una cámara se sienta correcta?** La respuesta no es trivial. Una buena cámara debe ser invisible para el usuario - cuando funciona correctamente, el jugador ni siquiera piensa en ella.

8.1. El Problema de la Cámara 3D

Controlar una cámara en un espacio tridimensional presenta varios desafíos fundamentales que no existen en juegos 2D. En 2D, la cámara simplemente se traslada en X e Y. En 3D, debemos manejar seis grados de libertad: traslación en tres ejes (X, Y, Z) y rotación en tres ejes (pitch, yaw, roll).

El primer desafío es la **representación de la orientación**. Los ángulos de Euler (pitch, yaw, roll) son intuitivos para los humanos, pero sufren del problema del gimbal lock y no permiten interpolación suave. Los cuaterniones resuelven estos problemas, pero son matemáticamente complejos.

El segundo desafío es el **control intuitivo**. El usuario mueve el mouse en 2D, pero esperamos que esto se traduzca en rotación natural de la cámara en 3D. Esta traducción no es obvia y requiere cuidadosa consideración de cómo mapear el input 2D al espacio 3D.

El tercer desafío es el **seguimiento de objetivos dinámicos**. Nuestro gato se mueve constantemente, salta entre edificios, y cambia de dirección impredeciblemente. La cámara debe mantener al gato en vista sin marear al usuario con movimientos bruscos.

8.2. Arquitectura de Cámara Orbital

Decidimos implementar una **cámara orbital**, también conocida como arcball camera. Este tipo de cámara rota alrededor de un punto fijo en el espacio - en nuestro caso, la posición del gato. Esta elección arquitectónica simplifica enormemente la implementación y proporciona controles intuitivos.

8.2.1. Coordenadas Esféricas

La clave del sistema orbital es pensar en la posición de la cámara usando coordenadas esféricas en lugar de coordenadas cartesianas. En lugar de especificar directamente X, Y, Z, especificamos:

- **distancia**: qué tan lejos está la cámara del objetivo
- **rotacion_y**: rotación horizontal (azimut)
- **rotacion_x**: rotación vertical (elevación)

La conversión de coordenadas esféricas a cartesianas se realiza usando trigonometría básica:

```
var offset = Vector3()
offset.x = sin(rotacion_y) * distancia * cos(rotacion_x)
offset.y = sin(rotacion_x) * distancia
offset.z = cos(rotacion_y) * distancia * cos(rotacion_x)
```

Esta formulación tiene ventajas importantes. Primero, es imposible que la cámara se "pierda" siempre está a una distancia fija del objetivo. Segundo, los controles son intuitivos: mover el mouse horizontalmente rota alrededor del eje Y, mover verticalmente cambia la elevación.

8.2.2. Restricciones de Movimiento

Una cámara completamente libre puede crear situaciones problemáticas. Por ejemplo, si permitimos rotación vertical completa, el usuario puede voltear la cámara completamente, resultando en controles invertidos confusos (de cabeza).

Esto sugiere la implementación de una cámara cinemática que este siempre alineada con el suelo. Es decir no inclinada o casi horizontal.

```
rotacion_x = clamp(rotacion_x, 0.1, PI/2.0)
```

Esta restricción mantiene la cámara entre casi horizontal (0.1 radianes) y completamente vertical ($\pi/2$ radianes), evitando orientaciones problemáticas donde el gato desaparecería de la vista o los controles se volverían contradictorios.

8.3. Suavizado de Movimiento

Los movimientos bruscos de cámara pueden causar náuseas y hacer que el juego se sienta poco pulido. Se puede entonces calcular múltiples capas de suavizado para crear movimiento fluido y natural.

8.3.1. Suavizado de Posición

En lugar de mover la cámara instantáneamente a su posición calculada, es preferible usar una interpolación exponencial. Pero, ¿qué es una interpolación exponencial?

```
var target_position = center + offset
var direction = target_position - global_transform.origin
global_transform.origin += direction * delta * 5
```

En el código de arriba se observa la implementación de esta, la cámara "sigue" al punto ideal gradualmente, pero nunca llegando exactamente. El factor multiplicativo nos ayuda a controlar la convergencia de la cámara. De este modo la velocidad de la cámara depende directamente de su distancia con el target.

8.3.2. Suavizado de Rotación con Cuaterniones

El suavizado de rotación es más complejo que el de posición porque las rotaciones no forman un espacio lineal. Usar interpolación lineal en ángulos de Euler puede producir rotaciones que pasan por orientaciones intermedias antinaturales.

La solución es usar interpolación esférica (slerp) de cuaterniones:

```
var target_transform = global_transform.looking_at(center, Vector3.UP)
var current_quat = global_transform.basis.get_rotation_quaternion()
var target_quat = target_transform.basis.get_rotation_quaternion()
var new_quat = current_quat.slerp(target_quat, delta * velocidad_suavizado)
global_transform.basis = Basis(new_quat)
```

El método `slerp` interpola suavemente entre dos cuaterniones, garantizando que la rotación siga el camino más corto en el espacio de rotaciones.

En este caso la implementación del `slerp` la realiza `godot` y no es necesario entender cuáles son los cálculos que se realizan directamente.

8.4. Control de Zoom Inteligente

El zoom no es simplemente acercar o alejar la cámara. Un buen sistema de zoom debe mantener el objetivo en vista.

8.4.1. Zoom Continuo vs Discreto

Se puede implementar un zoom continuo usando la rueda del mouse de la siguiente forma. La idea es simplemente modificar la variable distancia.

```
elif event.button_index == MOUSE_BUTTON_WHEEL_UP:
    distancia = max(zoom_min, distancia - zoom_speed)
elif event.button_index == MOUSE_BUTTON_WHEEL_DOWN:
    distancia = min(zoom_max, distancia + zoom_speed)
```

Los límites mínimo y máximo previenen que el usuario se acerque tanto que atraviese la geometría del gato, o se aleje tanto que pierda completamente el contexto visual.

Note que la distancia solo modifica el target de la cámara. Es decir que el suavizado anterior ya cumple la labor de dar un efecto profesional. No es necesario ser más complejos en este ámbito.

Capítulo 9

Sistema de Efectos Visuales

La importancia de los efectos visuales radica en comunicarnos con el jugador. El jugador debe tener una leve noción de hacia donde está yendo el gato.

9.1. El Problema de la Retroalimentación Visual

Cuando un usuario hace clic en el mundo 3D, varias cosas deben suceder simultáneamente. El gato debe recibir la instrucción de moverse hacia esa posición, pero igualmente importante, el usuario debe recibir **confirmación visual inmediata** de que su acción fue registrada y dónde.

Este problema es más complejo de lo que parece inicialmente. El clic del mouse ocurre en coordenadas 2D de pantalla, pero debe traducirse a una posición 3D en el mundo. Además, esta posición 3D puede estar en cualquier superficie con cualquier orientación, no necesariamente horizontal.

9.1.1. Raycast 3D desde Input 2D

El primer paso es convertir la posición 2D del mouse en una posición 3D del mundo. Esto se logra mediante un rayo tirado desde el click hasta el infinito en dirección de la cámara.

Los métodos `project_ray_origin` y `project_ray_normal` de Godot manejan la matemática compleja de convertir coordenadas de pantalla en rayos 3D considerando la matriz de proyección de la cámara.

9.2. Explosiones de Click: Diseño Orientado a Superficies

Una explosión simple que ignora la orientación de la superficie se ve artificial. Los efectos convincentes deben responder a la geometría del mundo. Nuestras explosiones se orientan según la normal de la superficie donde ocurre el clic.

9.2.1. Cálculo de Base Ortonormal

Cuando el raycast detecta una superficie, obtenemos su normal - un vector perpendicular a la superficie. Para crear efectos orientados correctamente, necesitamos construir un sistema de coordenadas completo alrededor de esta normal. El algoritmo propuesto entonces puede lucir algo así:

```
func orientar_segun_normal(mesh_instance: MeshInstance3D, surf_normal:
    Vector3):
    var up_vector = surf_normal
    var forward_vector = Vector3.FORWARD

    # Si la normal es muy parecida al forward, usar otro vector de
    # referencia
    if abs(up_vector.dot(forward_vector)) > 0.95:
        forward_vector = Vector3.RIGHT

    # Crear la base ortonormal
    var right_vector = forward_vector.cross(up_vector).normalized()
    forward_vector = up_vector.cross(right_vector).normalized()

    mesh_instance.global_transform.basis = Basis(right_vector, up_vector,
        forward_vector)
```

Este algoritmo construye tres vectores perpendiculares entre sí que definen completamente la orientación del efecto. El vector **up** es la normal de la superficie, y los otros dos vectores se calculan para formar una base ortonormal completa.

9.2.2. Geometría de Explosión: Torus Expansivo

Elegimos un torus (dona) como la geometría base para las explosiones porque proporciona varias ventajas visuales:

- Tiene un hueco central que no obstruye la vista del punto de impacto
- Su forma circular se expande uniformemente en todas las direcciones
- Puede orientarse según la superficie usando la base ortonormal calculada

La animación del torus combina expansión geométrica con cambios de transparencia.

9.3. Sistema de Partículas Físicas

Las partículas complementan las explosiones agregando elementos dinámicos que interactúan con la gravedad y se dispersan naturalmente desde el punto de impacto.

9.3.1. Distribución Espacial Realista

Las partículas se distribuyen en un hemisferio alrededor de la normal de la superficie, simulando cómo se dispersarían fragmentos reales.

Las partículas en godot pueden ser manejadas como una lista de partículas. Coherentemente lo único que se debe hacer es modificar la aceleración y velocidad de estas partículas y hacerlas desaparecer pasado cierto tiempo.

9.3.2. Simulación Física Simplificada

Cada partícula mantiene su propia simulación física básica con gravedad y tiempo de vida limitado:

```
func actualizar(delta: float) -> bool:
    tiempo_vida -= delta

    if tiempo_vida <= 0:
        return false

    velocidad.y -= 20.0 * delta # Gravedad
    posicion += velocidad * delta
    mesh.position = posicion

    # Fade out gradual
    var alpha = tiempo_vida / tiempo_vida_max
    material.albedo_color.a = alpha * color_inicial.a
```

La gravedad constante (20.0 unidades/segundo²) se aplica solo al componente Y de la velocidad, mientras que los componentes X y Z mantienen su momentum inicial, creando trayectorias parabólicas naturales.

Bibliografía

- [1] Godot Engine Documentation Team. *Godot 4.0 Documentation*. Godot Engine, 2023. <https://docs.godotengine.org/en/stable/>
- [2] Mojang Studios. *Minecraft*. Microsoft, 2011. Videojuego.
- [3] Blender Foundation. *Blender 3D Creation Suite*. Blender Foundation, 2023. <https://www.blender.org/>
- [4] Voronoi, Georgy. *Nouvelles applications des paramètres continus à la théorie des formes quadratiques*. Journal für die reine und angewandte Mathematik, 1908.
- [5] Delaunay, Boris. *Sur la sphère vide*. Bulletin de l'Académie des Sciences de l'URSS, 1934.